

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل نهم: Cluster analysis: advanced methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 9.1 Probabilistic model-based clustering
 - 9.1.1 Fuzzy clusters
 - 9.1.2 Probabilistic model-based clusters
 - 9.1.3 Expectation-maximization algorithm
- 9.2 Clustering high-dimensional data
 - 9.2.1 Why is clustering high-dimensional data challenging?
 - 9.2.2 Axis-parallel subspace approaches
 - 9.2.3 Arbitrarily oriented subspace approaches
- 9.3 Biclustering
 - 9.3.1 Why and where is biclustering useful?
 - 9.3.2 Types of biclusters
 - 9.3.3 Biclustering methods
 - 9.3.4 Enumerating all biclusters using MaPle
- 9.4 Dimensionality reduction for clustering
 - 9.4.1 Linear dimensionality reduction methods for clustering
 - 9.4.2 Nonnegative matrix factorization (NMF)
 - 9.4.3 Spectral clustering
- 9.5 Clustering graph and network data
 - 9.5.1 Applications and challenges
 - 9.5.2 Similarity measures
 - 9.5.3 Graph clustering methods
- 9.6 Semisupervised clustering
 - 9.6.1 Semisupervised clustering on partially labeled data
 - 9.6.2 Semisupervised clustering on pairwise constraints
 - 9.6.3 Other types of background knowledge for semisupervised clustering

Section 9.1

9.1.1. Fuzzy clusters

خوشه‌بندی فازی (Fuzzy Clustering)

خوشه‌بندی فازی (Fuzzy clustering) یک گسترش از روش‌های خوشه‌بندی سنتی است که به نقاط داده‌ای اجازه می‌دهد به طور همزمان به چندین خوشه، هر کدام با یک درجه عضویت (degree of membership)، تعلق داشته باشند. برخلاف خوشه‌بندی سخت (hard clustering) که در آن یک نقطه به طور انحصاری به یک خوشه واحد تعلق دارد، خوشه‌بندی فازی یک درجه عدم قطعیت را معرفی می‌کند. مدل اصلی مورد استفاده برای خوشه‌بندی فازی، الگوریتم فازی سی-مینز (Fuzzy C-Means - FCM) است.

در FCM، هر نقطه داده دارای مقادیر عضویت (membership values) در تمام خوشه‌ها است که به صورت یک ماتریس عضویت فازی (fuzzy membership matrix) نمایش داده می‌شود. این مقادیر عضویت بین ۰ و ۱ متغیر هستند که نشان‌دهنده احتمال تعلق یک نقطه داده‌ای به یک خوشه خاص است. خوشه‌بندی فازی به ویژه زمانی مفید است که نقاط داده‌ای عضویت جزئی (partial membership) به چندین خوشه را نشان می‌دهند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی فازی با استفاده از الگوریتم Fuzzy C-Means:

```
In [ ]: pip install scikit-learn-extra
```

```
In [5]: # وارد کردن کتابخانه‌های مورد نیاز
import numpy as np
from sklearn.datasets import make_blobs
from sklearn_extra.cluster import KMedoids
import matplotlib.pyplot as plt

# K-Medoids با مقداردهی اولیه از طریق Fuzzy C-Means تعریف تابع خوشه‌بندی
def fuzzy_c_means(X, n_clusters, m=2, max_iter=100, tol=1e-4):
    # K-Medoids مقداردهی اولیه مراکز خوشه با استفاده از الگوریتم 1.
    kmedoids = KMedoids(n_clusters=n_clusters, random_state=42)
    kmedoids.fit(X)
    centers = kmedoids.cluster_centers_

    for _ in range(max_iter):
        # 2. محاسبه فاصله بین هر نقطه و مراکز خوشه
        distances = np.linalg.norm(X[:, None] - centers[None, :], axis=2) + 1e-10 # جلوگیری از تقسیم بر صفر

        # 3. محاسبه ماتریس عضویت فازی (Fuzzy Partition Matrix)
        membership_matrix = 1.0 / distances ** (2 / (m - 1))
        membership_matrix = membership_matrix / np.sum(membership_matrix, axis=1, keepdims=True)

        # 4. به‌روزرسانی مراکز خوشه بر اساس وزن عضویت‌ها
        new_centers = (membership_matrix.T @ X) / np.sum(membership_matrix, axis=0)[:, None]

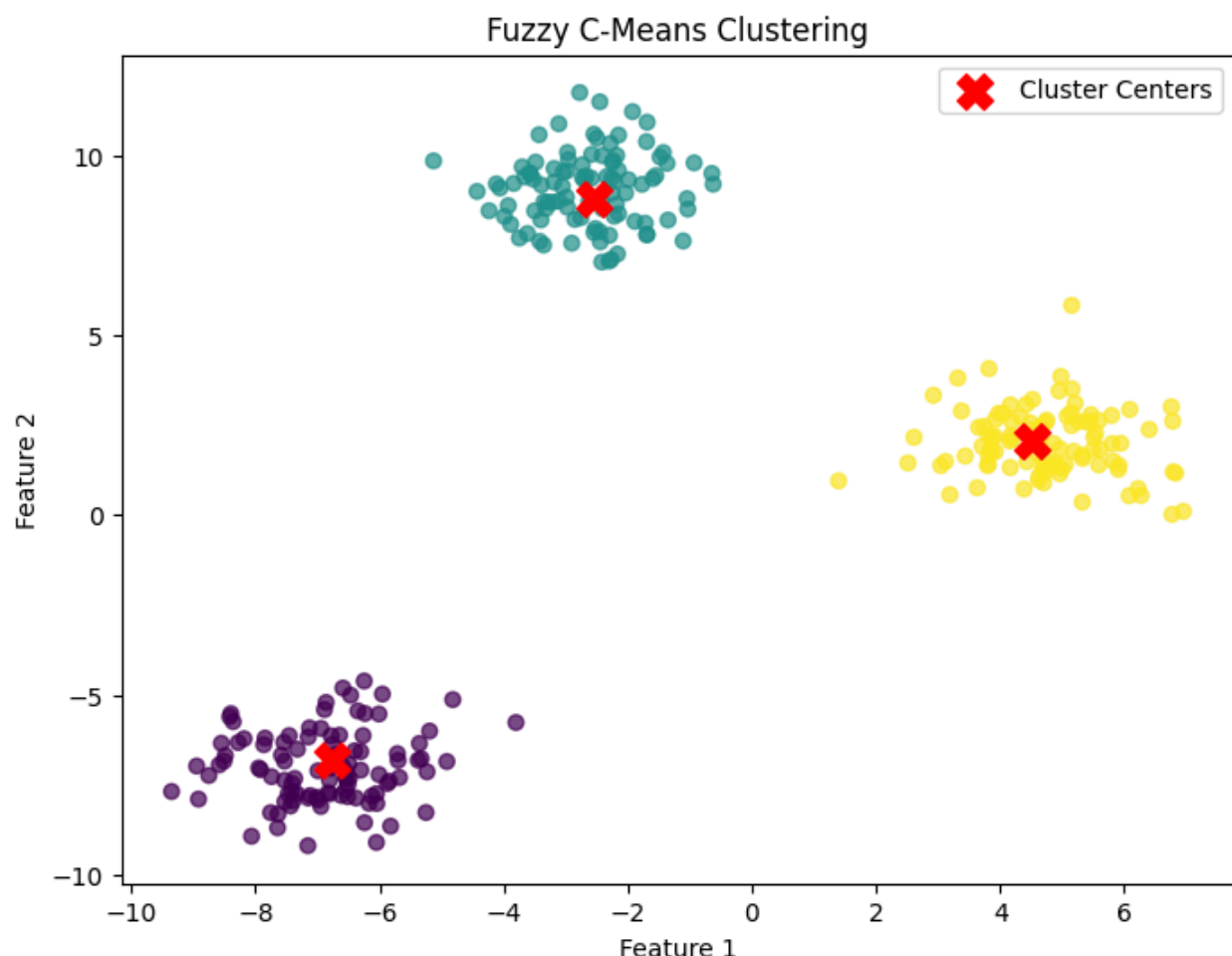
        # 5. بررسی شرط همگرایی
        if np.linalg.norm(new_centers - centers) < tol:
            break
        centers = new_centers

    # 6. تخصیص برچسب نهایی بر اساس بیشترین عضویت
    labels = np.argmax(membership_matrix, axis=1)
    return labels, centers, membership_matrix

# تولید داده‌های مصنوعی با سه خوشه
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# روی داده‌ها Fuzzy C-Means اعمال الگوریتم
n_clusters = 3
fuzzy_labels, fuzzy_centers, U = fuzzy_c_means(X, n_clusters)

# رسم نتایج خوشه‌بندی
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=fuzzy_labels, cmap='viridis', alpha=0.7)
plt.scatter(fuzzy_centers[:, 0], fuzzy_centers[:, 1], c='red', marker='X', s=200, label='Cluster Centers')
plt.title('Fuzzy C-Means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون ✨ 📊

در این مثال، الگوریتم خوشه‌بندی فازی **C-Means** برای گروه‌بندی داده‌های مصنوعی به کار گرفته شده است. برخلاف خوشه‌بندی سخت (مانند K-Means)، در این روش هر نقطه می‌تواند به چند خوشه با درجات مختلف تعلق داشته باشد:

- ابتدا داده‌های مصنوعی با سه خوشه مشخص تولید شده‌اند تا سناریوی خوشه‌بندی شبیه‌سازی شود.
- برای مقداردهی اولیه مراکز خوشه‌ها، از الگوریتم **K-Medoids** استفاده شده که نسبت به K-Means مقاوم‌تر به داده‌های پرت است.
- در هر تکرار، فاصله بین نقاط و مراکز خوشه محاسبه شده و سپس **ماتریس عضویت فازی** ساخته می‌شود که نشان می‌دهد هر نقطه با چه درجه‌ای به هر خوشه تعلق دارد.
- مراکز خوشه‌ها بر اساس وزن عضویت‌ها به‌روزرسانی می‌شوند تا زمانی که تغییرات مراکز کمتر از آستانه همگرایی شود.
- در پایان، برچسب هر نقطه بر اساس بیشترین عضویت تعیین شده و نتایج خوشه‌بندی به صورت تصویری نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی فازی**، ساختارهای پیچیده‌تری از داده‌ها را مدل‌سازی کرد. این روش به‌ویژه در کاربردهایی که مرز بین خوشه‌ها واضح نیست یا داده‌ها دارای هم‌پوشانی هستند، بسیار مؤثر است.

9.1.2. Probabilistic model-based clusters

📊 خوشه‌بندی مبتنی بر مدل احتمالی (Probabilistic Model-Based Clustering)

خوشه‌بندی مبتنی بر مدل احتمالی شامل استفاده از مدل‌های آماری برای توصیف توزیع اصلی

داده‌ها و شناسایی خوشه‌ها بر اساس پارامترهای این مدل‌ها است. یک رویکرد رایج، مدل ترکیبی گوسی (Gaussian Mixture Model - GMM) است، که در آن فرض می‌شود نقاط داده‌ای از ترکیبی از چندین توزیع گوسی تولید شده‌اند.

در خوشه‌بندی مبتنی بر GMM، هر خوشه با یک توزیع گوسی مرتبط است که با میانگین (mean)، ماتریس کوواریانس (covariance matrix) و وزن آن مشخص می‌شود. این مدل امکان انعطاف‌پذیری بیشتری در درک شکل‌های پیچیده خوشه را فراهم می‌کند و یک توزیع احتمال بر روی انتساب خوشه‌ها ارائه می‌دهد. الگوریتم Expectation-Maximization - EM اغلب برای تخمین پارامترها در GMM‌ها به کار گرفته می‌شود.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی مبتنی بر مدل احتمالی با استفاده از مدل ترکیبی گوسی:

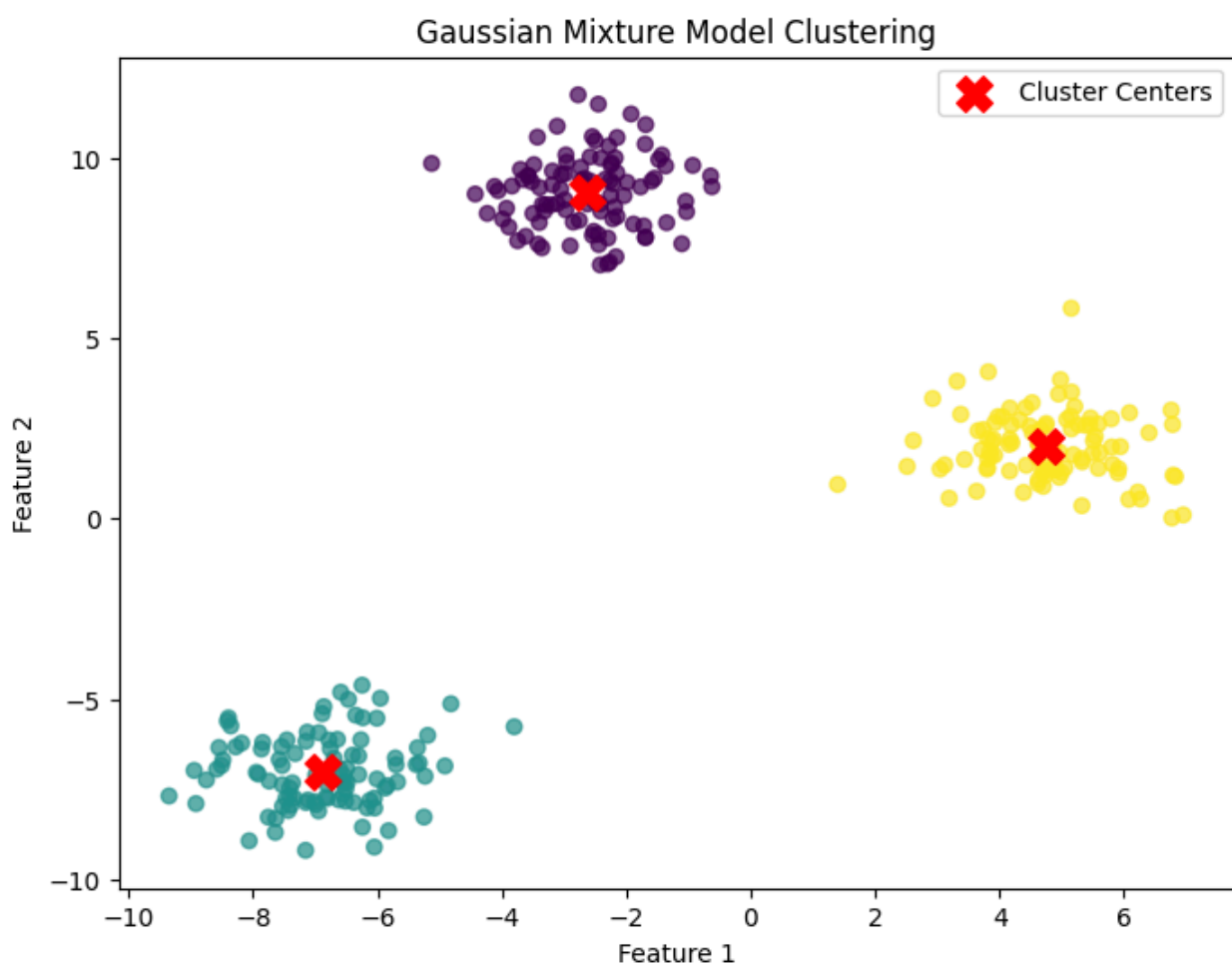
```
In [4]: from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
import numpy as np

# --- تولید داده‌های مصنوعی با ۳ خوشه ---
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# --- با ۳ مؤلفه GMM اعمال مدل آماری ---
n_components = 3
gmm = GaussianMixture(n_components=n_components, random_state=42)
gmm.fit(X)

# --- محاسبه احتمال تعلق هر نمونه به هر خوشه ---
probabilities = gmm.predict_proba(X)
labels = np.argmax(probabilities, axis=1) # انتخاب خوشه با بیشترین احتمال

# --- رسم نمودار پراکندگی داده‌ها و مراکز خوشه‌ها ---
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.scatter(gmm.means[:, 0], gmm.means[:, 1], c='red', marker='X', s=200, label='Cluster Centers')
plt.title('Gaussian Mixture Model Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از مدل **Gaussian Mixture Model (GMM)** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. برخلاف K-Means که هر نقطه را به یک خوشه خاص اختصاص می‌دهد، GMM به هر نمونه یک **احتمال** **تعلق**

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند.
 - مدل GMM با سه مؤلفه آموزش دیده و پارامترهای آماری هر خوشه (میانگین و کوواریانس) برآورد شده‌اند.
 - برای هر نمونه، احتمال تعلق به هر خوشه محاسبه شده و بر اساس بیشترین احتمال، برچسب خوشه تعیین شده است.
 - در نمودار، نقاط داده با رنگ خوشه‌ها و مراکز خوشه‌ها با علامت **X قرمز** نمایش داده شده‌اند.
- این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **مدل‌های آماری احتمالاتی**، خوشه‌بندی دقیق‌تری انجام داد. GMM به‌ویژه در داده‌هایی که خوشه‌ها هم‌پوشانی دارند یا شکل آن‌ها غیر کروی است، عملکرد بهتری نسبت به K-Means دارد.

9.1.3. Expectation-maximization algorithm

الگوریتم Expectation-Maximization (EM)

الگوریتم Expectation-Maximization (EM) یک چارچوب کلی برای یافتن تخمین‌های حداکثر درست‌نمایی (maximum likelihood estimates) از پارامترها در مدل‌های احتمالی (probabilistic models) با متغیرهای نهان (latent variables) است. در زمینه خوشه‌بندی مبتنی بر مدل احتمالی (probabilistic model-based clustering)، الگوریتم EM اغلب برای تخمین پارامتر در مدل‌های ترکیبی گوسی (Gaussian Mixture Models - GMMs) استفاده می‌شود.

در اینجا یک نمای کلی از الگوریتم EM در زمینه خوشه‌بندی مبتنی بر GMM آورده شده است:

- **گام امید (Expectation Step - E-step):** مقدار مورد انتظار (expected value) متغیرهای نهان (انتساب خوشه‌ها) را با توجه به داده‌های مشاهده شده و تخمین‌های فعلی پارامتر محاسبه کنید.
- **گام حداکثرسازی (Maximization Step - M-step):** پارامترهای توزیع‌های گوسی (میانگین، کوواریانس و وزن‌ها) را بر اساس داده‌های مشاهده شده و مقادیر مورد انتظار به دست آمده در گام E-step به‌روزرسانی کنید.
- **تکرار (Iteration):** گام‌های E و M را تا رسیدن به همگرایی (convergence)، جایی که تغییر در درست‌نمایی لگاریتمی (log-likelihood) یا پارامترها کمتر از یک آستانه از پیش تعریف شده باشد، تکرار کنید.

الگوریتم EM به صورت تکراری تخمین‌های پارامتر را اصلاح می‌کند و درست‌نمایی داده‌های مشاهده شده را با توجه به مدل حداکثر می‌سازد.

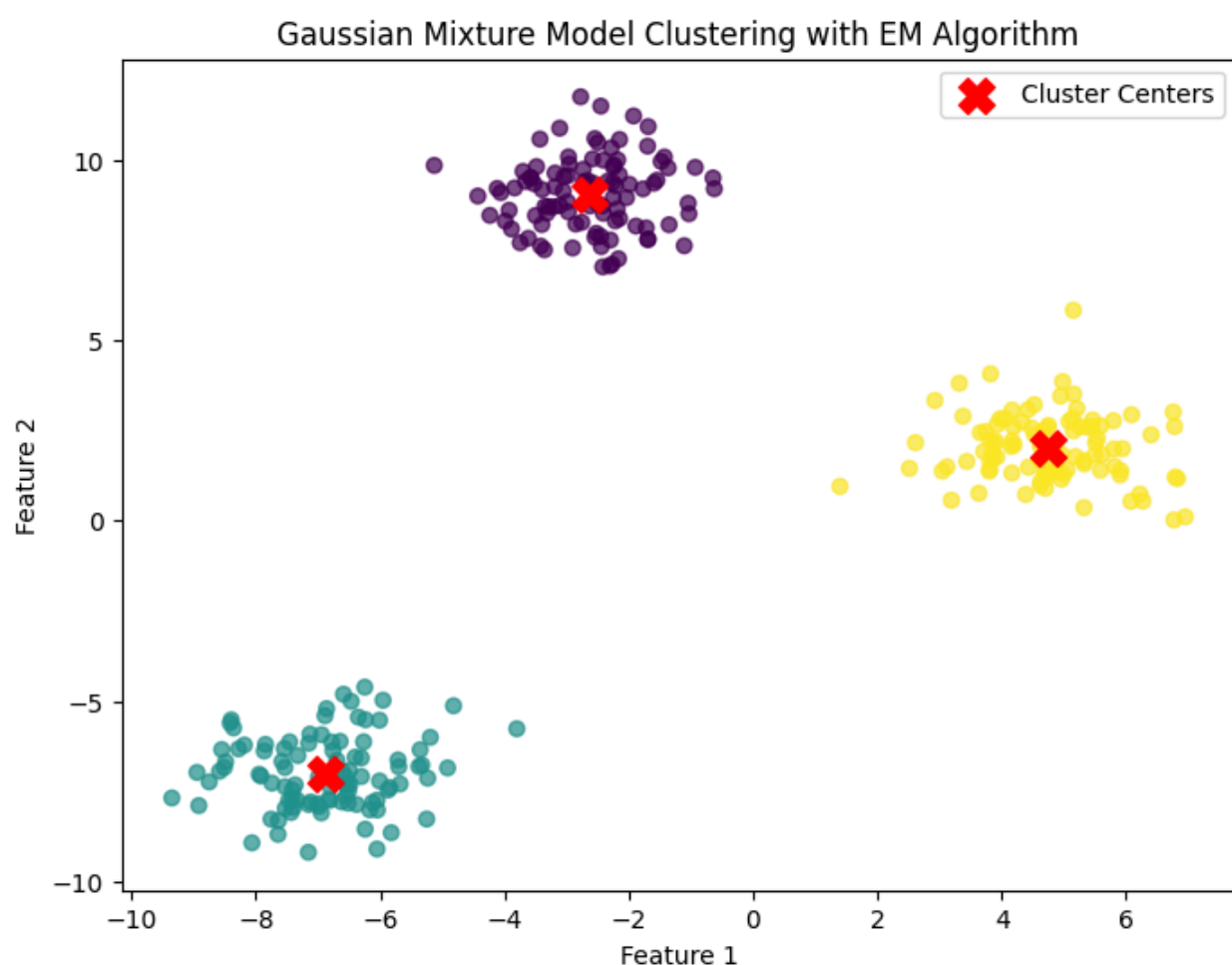
مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای الگوریتم Expectation-Maximization اعمال شده بر روی مدل‌های ترکیبی گوسی:

```
In [8]: from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
import numpy as np
```

```
# --- تولید داده‌های مصنوعی با ۳ خوشه ---
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# --- EM با الگوریتم GMM اعمال مدل آماری ---
n_components = 3
gmm = GaussianMixture(n_components=n_components, random_state=42)
gmm.fit(X)
labels = gmm.predict(X) # برچسب خوشه‌ها بر اساس بیشترین احتمال تعلق

# --- رسم نمودار پراکندگی داده‌ها و مراکز خوشه‌ها ---
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.scatter(gmm.means_[:, 0], gmm.means_[:, 1], c='red', marker='X', s=200, label='Cluster Centers')
plt.title('Gaussian Mixture Model Clustering with EM Algorithm')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از مدل **Gaussian Mixture Model (GMM)** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. این مدل با استفاده از الگوریتم **Expectation-Maximization (EM)** پارامترهای آماری هر خوشه را تخمین می‌زند:

- ابتدا داده‌هایی با سه خوشه تولید شده‌اند که به صورت تصادفی در فضای دوبعدی پخش شده‌اند.
- مدل GMM با سه مؤلفه آموزش دیده و برای هر نمونه، احتمال تعلق به هر خوشه محاسبه شده است.
- برچسب نهایی هر نمونه بر اساس بیشترین احتمال تعلق تعیین شده است.
- در نمودار، نقاط داده با رنگ خوشه‌ها و مراکز خوشه‌ها با علامت **X قرمز** نمایش داده شده‌اند.

این روش خوشه‌بندی مبتنی بر مدل‌های آماری است و نسبت به K-Means انعطاف‌پذیری بیشتری دارد، به‌ویژه زمانی که خوشه‌ها هم‌پوشانی دارند یا شکل آن‌ها غیر کروی است.

Section 9.2

9.2.2. Axis-parallel subspace approaches

خوشه‌بندی زیرفضای موازی با محور (Axis-parallel subspace clustering)

خوشه‌بندی داده‌های با ابعاد بالا چالش‌های منحصربه‌فردی را به همراه دارد، مانند نفرین ابعاد (curse of dimensionality) و وجود ویژگی‌های نامرتب یا اضافی. رویکردهای زیرفضا (subspace) موازی با محور (Axis-parallel) با هدف رسیدگی به این چالش‌ها از طریق شناسایی زیرفضاها در فضای ویژگی با ابعاد بالا که در آن خوشه‌ها واضح‌تر هستند، عمل می‌کنند.

ویژگی‌های کلیدی رویکردهای زیرفضای موازی با محور عبارتند از:

- **انتخاب زیرمجموعه ویژگی (Feature Subset Selection):** این روش‌ها شامل انتخاب یک زیرمجموعه از ویژگی‌ها یا ابعاد است که برای خوشه‌بندی آموزنده‌ترین هستند، و ابعاد نامرتب یا اضافی را کنار می‌گذارند.
- **خوشه‌بندی موازی با محور (Axis-Parallel Clustering):** خوشه‌بندی در زیرفضای انتخاب شده با استفاده از روش‌هایی انجام می‌شود که به هندسه داده‌ها در امتداد هر محور حساس هستند.
- **نمایش و بصری‌سازی (Projection and Visualization):** خوشه‌بندی زیرفضا اغلب شامل نمایش داده‌ها بر روی زیرفضای انتخاب شده برای بصری‌سازی و تفسیر بهتر است.

تکنیک‌های رایج برای خوشه‌بندی زیرفضای موازی با محور شامل خوشه‌بندی زیرفضا بر اساس k-means یا خوشه‌بندی مبتنی بر چگالی (density-based clustering) در داخل زیرفضاها است.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی زیرفضای موازی با محور با استفاده از الگوریتم k-means را ارائه دهیم:

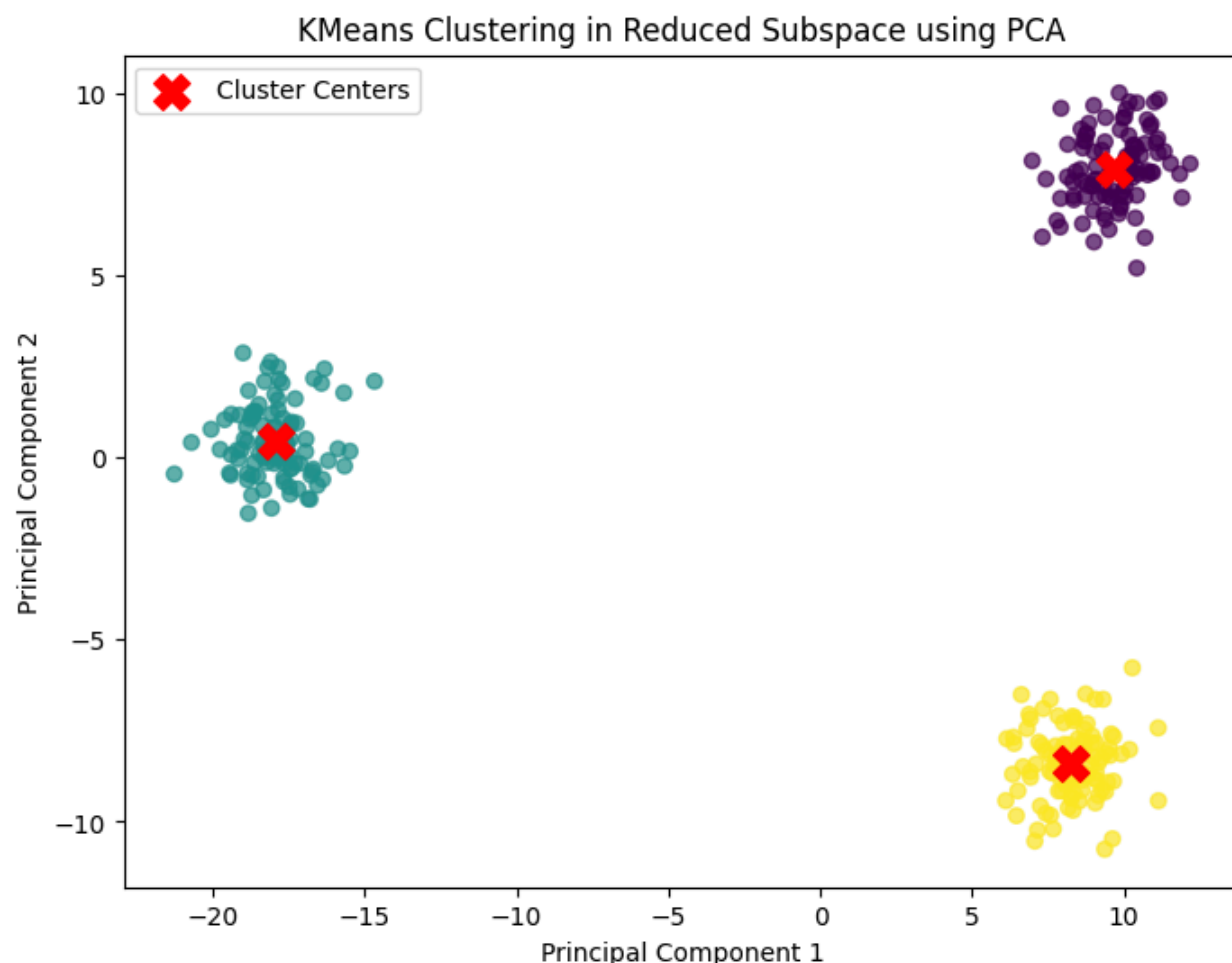
```
In [9]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# تولید داده‌های مصنوعی با ابعاد بالا (۱۰ ویژگی) و ۳ خوشه
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0, n_features=10)

# برای کاهش ابعاد به ۲ مؤلفه اصلی (PCA) اعمال تحلیل مؤلفه‌های اصلی
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# روی فضای کاهش‌یافته KMeans اعمال خوشه‌بندی
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X_pca)

# رسم نتایج خوشه‌بندی در فضای دو مؤلفه اصلی
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], c='red', marker='X', s=200, label='Cluster Centers')
plt.title('KMeans Clustering in Reduced Subspace using PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، ترکیب دو تکنیک قدرتمند یعنی **تحلیل مؤلفه‌های اصلی (PCA)** و **خوشه‌بندی KMeans** برای تحلیل داده‌های مصنوعی با ابعاد بالا به کار گرفته شده است:

- ابتدا داده‌های مصنوعی با ۱۰ ویژگی و ۳ خوشه تولید شده‌اند تا سناریوی خوشه‌بندی در فضای چندبعدی شبیه‌سازی شود.
- برای کاهش پیچیدگی و امکان تصویرسازی، از **PCA** برای کاهش ابعاد به ۲ مؤلفه اصلی استفاده شده است. این مؤلفه‌ها بیشترین واریانس داده را حفظ می‌کنند.
- سپس الگوریتم **KMeans** روی داده‌های کاهش‌یافته اعمال شده تا خوشه‌ها در فضای جدید شناسایی شوند.
- در پایان، نقاط داده با رنگ‌بندی بر اساس خوشه‌ها و مراکز خوشه با علامت قرمز رسم شده‌اند تا ساختار خوشه‌بندی به‌وضوح قابل مشاهده باشد.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **کاهش ابعاد**، داده‌های پیچیده را ساده‌سازی کرد و سپس با **خوشه‌بندی**، الگوهای پنهان در آن‌ها را کشف نمود. این ترکیب در بسیاری از کاربردهای عملی مانند تحلیل رفتار مشتری، فشردگی داده و تصویرسازی داده‌های چندبعدی بسیار مؤثر است.

9.2.3. Arbitrarily oriented subspace approaches

رویکردهای زیرفضای با جهت‌گیری دلخواه (Arbitrarily oriented subspace approaches)

رویکردهای زیرفضای با جهت‌گیری دلخواه (Arbitrarily oriented subspace approaches) به چالش‌های خوشه‌بندی داده‌های با ابعاد بالا با شناسایی زیرفضاهایی که لزوماً با محورهای

فضای ویژگی اصلی هم‌راستا نیستند، می‌پردازند. برخلاف رویکردهای موازی با محور، این روش‌ها زیرفضاها را با جهت‌گیری‌های دلخواه در نظر می‌گیرند که انعطاف‌پذیری بیشتری را در ثبت روابط پیچیده در داده‌ها فراهم می‌سازد.

ویژگی‌های کلیدی رویکردهای زیرفضای با جهت‌گیری دلخواه عبارتند از:

- تعریف زیرفضا (Subspace Definition):** این روش‌ها امکان تعریف زیرفضاها را بر اساس ترکیب‌های خطی (linear combinations) از ویژگی‌های اصلی فراهم می‌سازند و شناسایی خوشه‌ها را در زیرفضاهای غیرموازی با محور ممکن می‌سازند.
- تکنیک‌های خوشه‌بندی زیرفضا (Subspace Clustering Techniques):** تکنیک‌هایی مانند خوشه‌بندی زیرفضا بر اساس انتشار وابستگی (affinity propagation) یا خوشه‌بندی طیفی (spectral clustering) اغلب برای شناسایی خوشه‌ها در داخل زیرفضاهای تعریف شده به کار گرفته می‌شوند.
- مقاومت در برابر افزونگی ویژگی (Robustness to Feature Redundancy):** رویکردهای زیرفضای با جهت‌گیری دلخواه عموماً در برابر افزونگی ویژگی مقاوم‌تر هستند، که آن‌ها را برای مجموعه‌های داده با ابعاد بالا و ویژگی‌های همبسته (correlated features) مناسب می‌سازد.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی زیرفضای با جهت‌گیری دلخواه با استفاده از الگوریتم خوشه‌بندی طیفی (Spectral Clustering) ارائه دهیم:

```
In [30]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn.datasets import make_blobs
from sklearn.decomposition import PCA
from sklearn.cluster import SpectralClustering
import matplotlib.pyplot as plt
import numpy as np

# مرحله ۱: تولید داده‌های مصنوعی با ابعاد بالا
X, _ = make_blobs(
    n_samples=300,
    centers=3,
    n_features=10,
    random_state=42,
    cluster_std=1.0
)

# برای تصویرسازی PCA مرحله ۲: کاهش ابعاد با
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# (هسته گاوسی) RBF از نوع affinity خوشه‌بندی طیفی با a: مرحله ۳
spectral_rbf = SpectralClustering(
    n_clusters=3,
    affinity="rbf",
    random_state=42
)
labels_rbf = spectral_rbf.fit_predict(X)

# Nearest Neighbors از نوع affinity خوشه‌بندی طیفی با b: مرحله ۳

# affinity="nearest_neighbors" یعنی:
# - نزدیک‌ترین همسایه‌ها ساخته می‌شود k روی graph یک -
# - محاسبه می‌شود graph روی این eigenvectors -
# technical/نکته علمی:
# - Spectral Clustering روی یک گراف connected می‌کند
# - warning درست محاسبه نمیشن و eigenvectors، باشد graph disconnected اگر -
# n_neighbors: انتخاب
# - بشود graph connected باید به اندازه‌ای باشد که -
# - rule of thumb: sqrt(n_samples) تا log(n_samples) بین
# - تضمین بشود connectivity کمی بزرگ‌تر می‌گیریم تا practice در -
# trade-off:
# - connectivity بشود غرق کمی محلی اطلاعات محلی کمی غرق بشود connectivity -

spectral_nn = SpectralClustering(
    n_clusters=3,
    affinity="nearest_neighbors",
    n_neighbors=120, # تضمین اتصال گراف برای ۳۰۰ نمونه
```

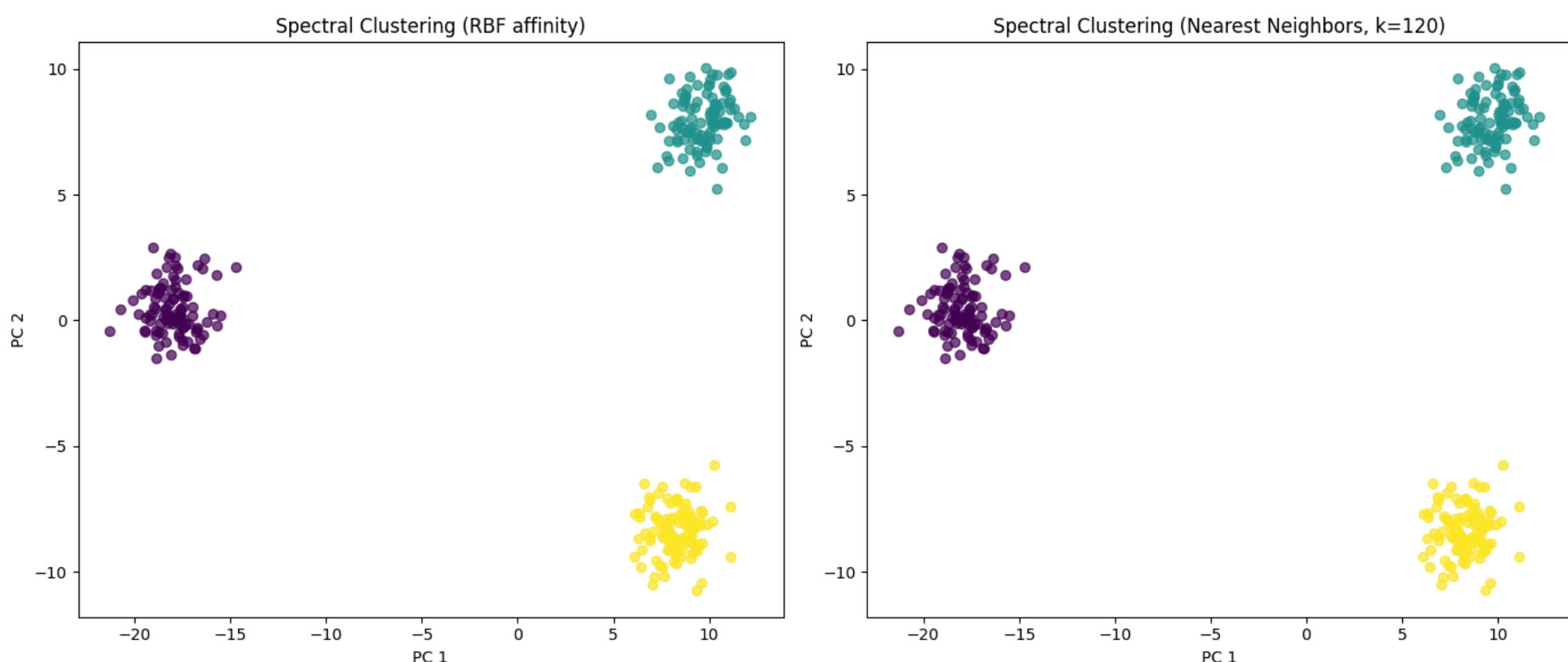
```
random_state=42
)
labels_nn = spectral_nn.fit_predict(X)

# مرحله ۴: رسم نتایج خوشه‌بندی در فضای کاهش‌یافته
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# رسم نتایج RBF
axes[0].scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=labels_rbf,
    cmap="viridis",
    alpha=0.7
)
axes[0].set_title("Spectral Clustering (RBF affinity)")
axes[0].set_xlabel("PC 1")
axes[0].set_ylabel("PC 2")

# رسم نتایج Nearest Neighbors
axes[1].scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=labels_nn,
    cmap="viridis",
    alpha=0.7
)
axes[1].set_title("Spectral Clustering (Nearest Neighbors, k=120)")
axes[1].set_xlabel("PC 1")
axes[1].set_ylabel("PC 2")

# بهبود چیدمان نمودارها
plt.tight_layout()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، الگوریتم **خوشه‌بندی طیفی (Spectral Clustering)** روی داده‌های مصنوعی با ابعاد بالا اعمال شده و نتایج آن با استفاده از **PCA** در فضای دوبعدی تصویرسازی شده‌اند. این روش برای شناسایی ساختارهای پیچیده و غیرخطی در داده‌ها بسیار مؤثر است:

- ابتدا داده‌های مصنوعی با ۱۰ ویژگی و ۳ خوشه تولید شده‌اند تا سناریوی خوشه‌بندی در فضای چندبعدی شبیه‌سازی شود. این مرحله مشابه بسیاری از مطالعات علمی و پروژه‌های صنعتی است که نیاز به شبیه‌سازی داده‌های high-dimensional دارند.
- برای کاهش ابعاد و امکان تصویرسازی، از **PCA** استفاده شده است. PCA مؤلفه‌های اصلی داده را استخراج می‌کند و واریانس بیشینه را حفظ می‌نماید تا ساختار اصلی داده‌ها از بین نرود. این مرحله هم در تحقیقات علمی و هم در کاربردهای صنعتی برای فهم بهتر داده‌ها ضروری است.
- دو نوع خوشه‌بندی طیفی اعمال شده‌اند:

▪ **RBF affinity**: با هسته گاوسی، مناسب برای داده‌های غیرخطی و خوشه‌های پیچیده است و در بسیاری از مقالات و پروژه‌های عملی استاندارد محسوب می‌شود.

▪ **Nearest Neighbors affinity**: ساخت گراف روی k نزدیک‌ترین همسایه‌ها و حفظ ساختار محلی داده‌ها. برای این روش، تعداد همسایه‌ها به گونه‌ای انتخاب شده که گراف متصل باشد تا eigenvectorها به درستی محاسبه شوند.

- انتخاب **n_neighbors** بر اساس تعداد نمونه‌ها انجام شده است. قاعده تقریبی: بین $\log(n_samples)$ تا $\sqrt{n_samples}$ و در عمل کمی بزرگ‌تر برای تضمین connectivity انتخاب می‌شود. این نکته بسیار مهم است چرا که Spectral Clustering روی گراف disconnected نتایج نادرست و هشدار می‌دهد.
- نتایج هر دو روش در فضای دوبعدی رسم شده‌اند تا تفاوت ساختار خوشه‌ها به وضوح قابل مشاهده باشد و تحلیل نتایج راحت‌تر شود.

این مثال نشان می‌دهد که چگونه می‌توان با ترکیب **کاهش ابعاد و خوشه‌بندی طیفی**، ساختارهای پنهان در داده‌های پیچیده را کشف و تحلیل کرد. این روش در کاربردهای واقعی مانند تحلیل تصاویر، داده‌های زیستی، شبکه‌های اجتماعی و تحلیل رفتار کاربران بسیار پرکاربرد است و هم در مقالات پژوهشی و هم در پروژه‌های صنعتی اهمیت بالایی دارد.

Section 9.3

9.3.2. Types of biclusters

خوشه‌بندی دوقطبی (Biclustering)

خوشه‌بندی دوقطبی (Biclustering) تکنیکی است که به صورت همزمان سطرها و ستون‌های یک مجموعه داده را خوشه‌بندی می‌کند و زیرماتریس‌هایی را شناسایی می‌کند که الگوهای منسجمی از خود نشان می‌دهند. چندین نوع خوشه دوقطبی بر اساس ماهیت الگوهایی که ثبت می‌کنند وجود دارد:

- **خوشه‌های دوقطبی با مقدار ثابت (Constant Value Biclusters)**: خوشه‌های دوقطبی که در آن‌ها تمام عناصر دارای یک مقدار ثابت هستند و نشان‌دهنده یکنواختی در داخل زیرماتریس شناسایی شده است.
- **خوشه‌های دوقطبی با جابجایی ثابت (Shift Biclusters)**: خوشه‌های دوقطبی که در آن‌ها مقادیر درون زیرماتریس یک جابجایی ثابت را نشان می‌دهند و الگوی کلی را حفظ می‌کنند، اما امکان تغییر را فراهم می‌سازند.
- **خوشه‌های دوقطبی افزایشی (Additive Biclusters)**: خوشه‌های دوقطبی که در آن‌ها مقادیر می‌توانند به عنوان مجموع یک ثابت و یک جابجایی مدل‌سازی شوند، و الگوهای ثابت و جابجایی را به طور همزمان ثبت می‌کنند.
- **خوشه‌های دوقطبی ضربی (Multiplicative Biclusters)**: خوشه‌های دوقطبی که در آن‌ها مقادیر می‌توانند به عنوان حاصل ضرب یک ثابت و یک جابجایی مدل‌سازی شوند، و الگوهای ثابت و مقیاس‌گذاری را ثبت می‌کنند.

درک نوع خوشه دوقطبی برای تفسیر معنای بیولوژیکی یا خاص دامنه در پس الگوهای شناسایی شده در داده‌ها ضروری است.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی دوقطبی با استفاده از کلاس Bicluster از کتابخانه sklearn ارائه دهیم:

```
In [33]: import numpy as np
from sklearn.cluster import SpectralBiclustering
import matplotlib.pyplot as plt

# --- تعریف تعداد خوشه‌های سطری و ستونی ---
n_row_clusters, n_col_clusters = 3, 3
block_size = 100 # اندازه هر بلوک

# --- ساخت الگوی شطرنجی پایه (ماتریس ۳×۳ از ۰ و ۱) ---
pattern = np.array([[0, 1, 0],
                    [1, 0, 1],
                    [0, 1, 0]])

# --- گسترش الگو با ضرب کرونکر برای ایجاد ماتریس بزرگ‌تر ---
data = np.kron(pattern, np.ones((block_size, block_size)))

# --- افزودن نویز نرمال به داده‌ها ---
noise = np.random.normal(0, 0.1, size=data.shape)
data_noisy = data + noise

# --- اعمال الگوریتم Spectral Biclustering ---
model = SpectralBiclustering(n_clusters=(n_row_clusters, n_col_clusters), random_state=42)
model.fit(data_noisy)

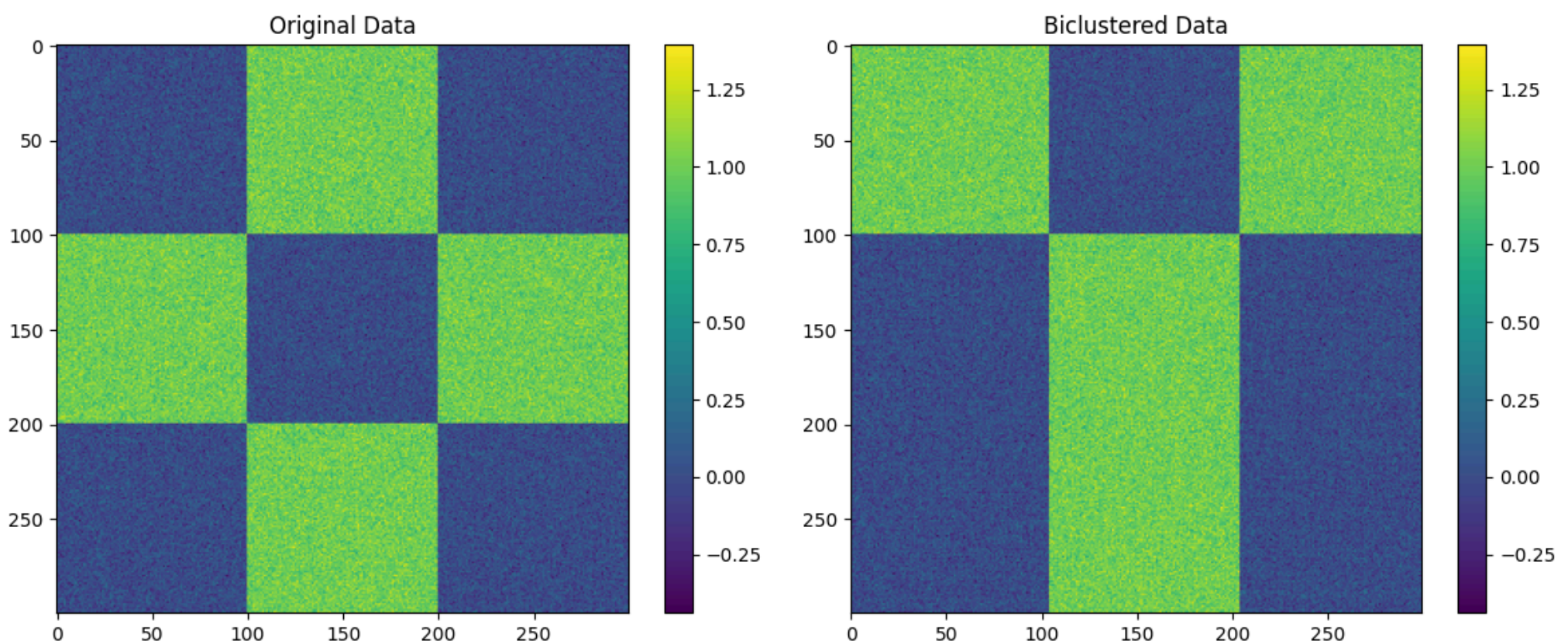
# --- دریافت برچسب‌های سطری و ستونی ---
rows, cols = model.row_labels_, model.column_labels_

# --- رسم داده‌ی اصلی و داده‌ی خوشه‌بندی‌شده ---
plt.figure(figsize=(12, 5))

# --- نمایش داده‌ی اصلی ---
plt.subplot(1, 2, 1)
plt.title("Original Data")
plt.imshow(data_noisy, cmap="viridis", aspect="auto")
plt.colorbar()

# --- نمایش داده‌ی خوشه‌بندی‌شده با مرتب‌سازی سطرها و ستون‌ها ---
plt.subplot(1, 2, 2)
plt.title("Biclustered Data")
plt.imshow(data_noisy[np.argsort(rows)][:, np.argsort(cols)], cmap="viridis", aspect="auto")
plt.colorbar()

plt.tight_layout()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Spectral Biclustering** برای شناسایی خوشه‌های هم‌زمان در سطرها و ستون‌های یک ماتریس داده استفاده شده است. این الگوریتم به‌ویژه برای داده‌هایی با ساختار شطرنجی یا بلوکی مناسب است:

- ابتدا یک الگوی شطرنجی 3×3 با استفاده از ضرب کرونکر گسترش داده شده تا ماتریسی بزرگ‌تر با ساختار بلوکی ایجاد شود.
- برای واقع‌گراتر شدن داده‌ها، نویز نرمال به ماتریس افزوده شده است.
- الگوریتم Spectral Biclustering اجرا شده و خوشه‌های سطری و ستونی استخراج شده‌اند.
- در نمودار اول، داده‌ی اصلی نمایش داده شده و در نمودار دوم، داده‌ها بر اساس خوشه‌های شناسایی‌شده مرتب شده‌اند تا ساختار بلوکی بهتر دیده شود.

این روش در تحلیل داده‌های ژنومیکس، توصیه‌گرها، و ماتریس‌های تعامل کاربرد دارد و می‌تواند الگوهای پنهان بین زیرمجموعه‌های سطرها و ستون‌ها را آشکار کند.

9.3.3. Biclustering methods

روش‌های خوشه‌بندی دوقطبی (Biclustering methods)

روش‌های خوشه‌بندی دوقطبی (Biclustering methods) برای کشف الگوها در مجموعه‌های داده‌ای طراحی شده‌اند که در آن‌ها زیرمجموعه‌هایی از سطرها و ستون‌ها رفتار مشابهی از خود نشان می‌دهند. این روش‌ها ابزاری قدرتمند برای کشف ساختارهای پیچیده در انواع مختلف داده‌ها فراهم می‌کنند. برخی از الگوریتم‌های خوشه‌بندی دوقطبی رایج عبارتند از:

- **خوشه‌بندی دوقطبی طیفی (Spectral Biclustering):** این روش از تجزیه مقادیر ویژه (eigenvalue decomposition) برای شناسایی خوشه‌های دوقطبی در داده‌ها استفاده می‌کند. این روش هم روابط سطر و هم ستون را در نظر می‌گیرد و به ویژه برای کشف خوشه‌های دوقطبی افزایشی (additive biclusters) مؤثر است.
- **خوشه‌بندی دوقطبی با مدل پلید (Plaid Model Biclustering):** مدل پلید (Plaid Model) داده‌ها را به عنوان یک سوپرپوزیشن از خوشه‌های دوقطبی افزایشی نمایش می‌دهد. خوشه‌های دوقطبی با تطبیق الگوهای سطر و ستون به صورت تکراری یافت می‌شوند.
- **خوشه‌بندی دوقطبی بیمکس (Bimax Biclustering):** بیمکس (Bimax) یک روش تجزیه ماتریس دودویی (binary matrix factorization) است که با هدف یافتن زیرماتریس‌هایی که تنها شامل ۰ و ۱ هستند، عمل می‌کند. این روش برای داده‌های دودویی مناسب است.
- **خوشه‌بندی دوقطبی حفظ‌کننده ترتیب (Order-Preserving Biclustering):** این روش با حفظ ترتیب نقاط داده در سطرها و ستون‌ها، خوشه‌های دوقطبی را شناسایی می‌کند و الگوها را بر اساس روابط یکنواخت (monotonic relationships) ثبت می‌کند.

هر روش خوشه‌بندی دوقطبی برای رسیدگی به ویژگی‌های خاص داده‌ها و انواعی از الگوهایی که انتظار می‌رود یافت شوند، طراحی شده است.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی دوقطبی با استفاده از کلاس SpectralBiclustering از کتابخانه sklearn:

```
from sklearn.cluster import SpectralBiclustering
import matplotlib.pyplot as plt
import numpy as np

# تولید داده‌های مصنوعی به صورت ماتریس شطرنجی با ۳ خوشه در ردیف و ۳ خوشه در ستون
data = make_checkerboard(shape=(300, 300), n_clusters=(3, 3), noise=10, random_state=42)[0]

# بررسی نوع و شکل داده‌ها
print(type(data), data.shape) # با ابعاد (300, 300) باشد numpy.ndarray خروجی باید

# اعمال الگوریتم خوشه‌بندی طیفی دوطرفه (Spectral Biclustering)
model = SpectralBiclustering(n_clusters=(3, 3), random_state=42)
model.fit(data)

# استخراج برچسب‌های خوشه برای ردیف‌ها و ستون‌ها
rows, cols = model.row_labels_, model.column_labels_

# مرتب‌سازی داده‌ها بر اساس برچسب‌های خوشه برای نمایش واضح‌تر بلوک‌ها
sorted_data = data[np.argsort(rows)][:, np.argsort(cols)]

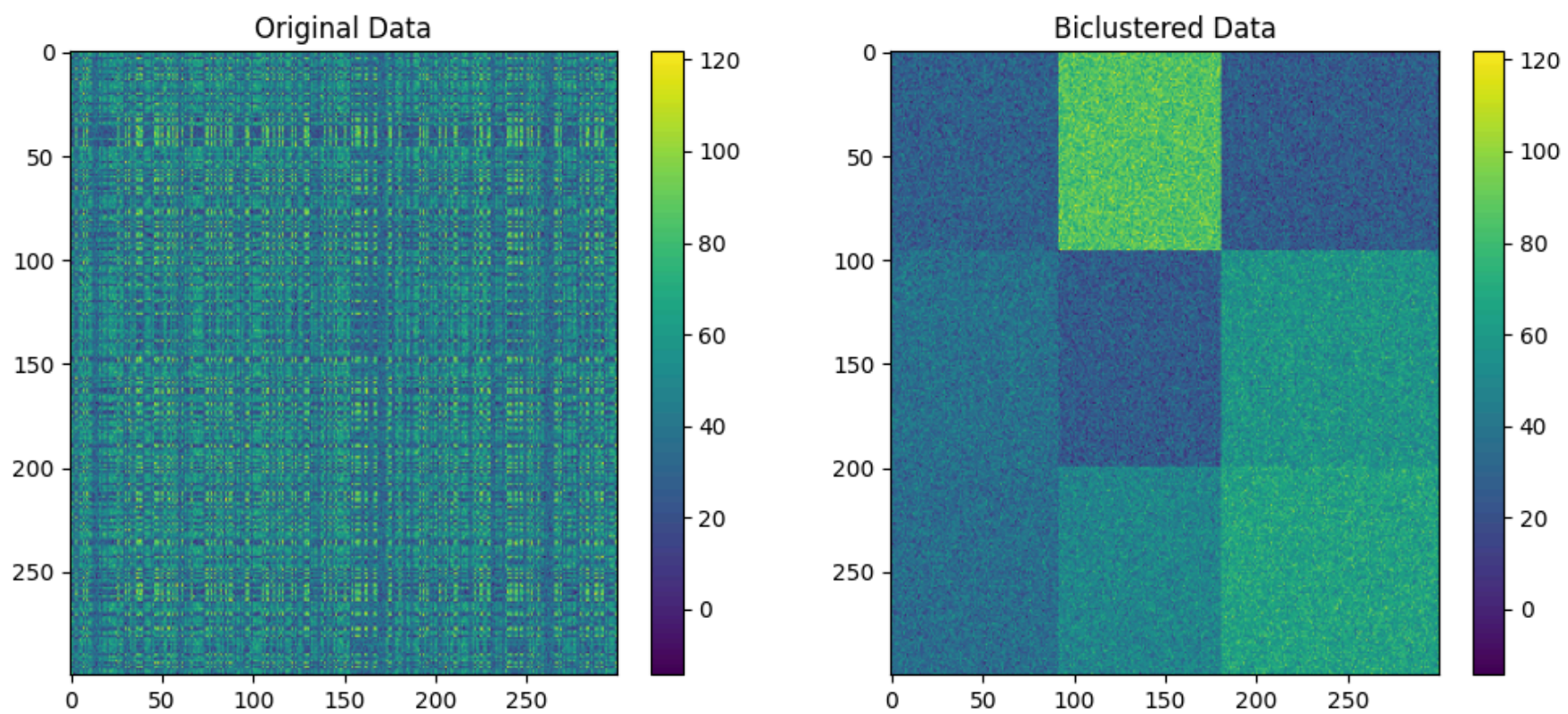
# رسم داده‌ها قبل و بعد از خوشه‌بندی
plt.figure(figsize=(12, 5))

# نمایش داده اولیه
plt.subplot(1, 2, 1)
plt.title('Original Data')
plt.imshow(data, cmap='viridis', aspect='auto')
plt.colorbar()

# نمایش داده مرتب‌شده بر اساس خوشه‌ها
plt.subplot(1, 2, 2)
plt.title('Biclustered Data')
plt.imshow(sorted_data, cmap='viridis', aspect='auto')
plt.colorbar()

plt.show()
```

<class 'numpy.ndarray'> (300, 300)



توضیحات مثال کد پایتون 📊🔗

در این مثال، از الگوریتم خوشه‌بندی طیفی دوطرفه (**Spectral Biclustering**) برای شناسایی ساختار بلوکی در داده‌های مصنوعی استفاده شده است. این الگوریتم به‌طور همزمان خوشه‌هایی در ردیف‌ها و ستون‌ها پیدا می‌کند:

- ابتدا داده‌ای به شکل ماتریس شطرنجی با ۳ خوشه در ردیف و ۳ خوشه در ستون تولید شده است. این نوع داده‌ها برای تست الگوریتم‌های biclustering بسیار مناسب هستند.
- الگوریتم **Spectral Biclustering** روی داده اعمال شده تا ساختارهای پنهان در هر دو محور شناسایی شوند. این الگوریتم از تجزیه طیفی برای یافتن الگوهای همزمان در ردیف‌ها و ستون‌ها استفاده می‌کند.
- برچسب‌های خوشه برای ردیف‌ها و ستون‌ها استخراج شده و داده‌ها بر اساس آن‌ها مرتب شده‌اند تا بلوک‌های خوشه‌ای به‌وضوح قابل مشاهده باشند.
- در نهایت، داده اولیه و داده خوشه‌بندی‌شده به صورت تصویری نمایش داده شده‌اند تا تفاوت ساختار قبل و بعد از biclustering مشخص شود.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی دوطرفه**، الگوهای پیچیده و هم‌زمان در ردیف‌ها و ستون‌های داده را کشف کرد. این روش در تحلیل داده‌های ژنومیک، توصیه‌گرها و ماتریس‌های تعامل بسیار کاربردی و مؤثر است.

Section 9.4

9.4.1. Linear dimensionality reduction methods for clustering

کاهش ابعاد خطی (Linear dimensionality reduction)

روش‌های کاهش ابعاد خطی (Linear dimensionality reduction methods) با هدف کاهش تعداد ویژگی‌ها در یک مجموعه داده در عین حفظ اطلاعات ضروری، عمل می‌کنند. این تکنیک‌ها به ویژه برای خوشه‌بندی داده‌های با ابعاد بالا مفید هستند، زیرا می‌توانند کارایی و تفسیرپذیری الگوریتم‌های خوشه‌بندی را افزایش دهند. برخی از روش‌های رایج کاهش ابعاد خطی برای خوشه‌بندی عبارتند از:

- تحلیل مؤلفه‌های اصلی (Principal Component Analysis - PCA):** PCA مجموعه‌ای از محورهای متعامد (orthogonal axes) (مؤلفه‌های اصلی) را شناسایی می‌کند که حداکثر واریانس را در داده‌ها ثبت می‌کنند. این روش برای کاهش ابعاد در عین حفظ ساختار کلی، مؤثر است.
- تحلیل تفکیک خطی (Linear Discriminant Analysis - LDA):** LDA با هدف حداکثر کردن جداسازی بین کلاس‌های مختلف در داده‌ها عمل می‌کند. این روش به ویژه زمانی برای خوشه‌بندی مفید است که اطلاعات کلاس در دسترس باشد.
- تجزیه ماتریس غیرمنفی (Non-Negative Matrix Factorization - NMF):** NMF ماتریس داده را به ماتریس‌های غیرمنفی تجزیه می‌کند و ویژگی‌های مبتنی بر بخش (parts-based) و قابل تفسیر را نمایش می‌دهد. این روش برای داده‌هایی با مقادیر غیرمنفی مناسب است.
- تحلیل مؤلفه‌های مستقل (Independent Component Analysis - ICA):** ICA به دنبال یافتن مؤلفه‌های مستقل (independent components) در داده‌ها است. این روش برای جداسازی سیگنال‌های مختلط (mixed signals) و شناسایی منابع زیربنایی مفید است.

این روش‌های کاهش ابعاد خطی می‌توانند قبل از خوشه‌بندی اعمال شوند تا نویز را کاهش داده، کارایی محاسباتی را بهبود بخشند و کیفیت انتساب خوشه‌ها را افزایش دهند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاهش ابعاد خطی با استفاده از PCA برای خوشه‌بندی:

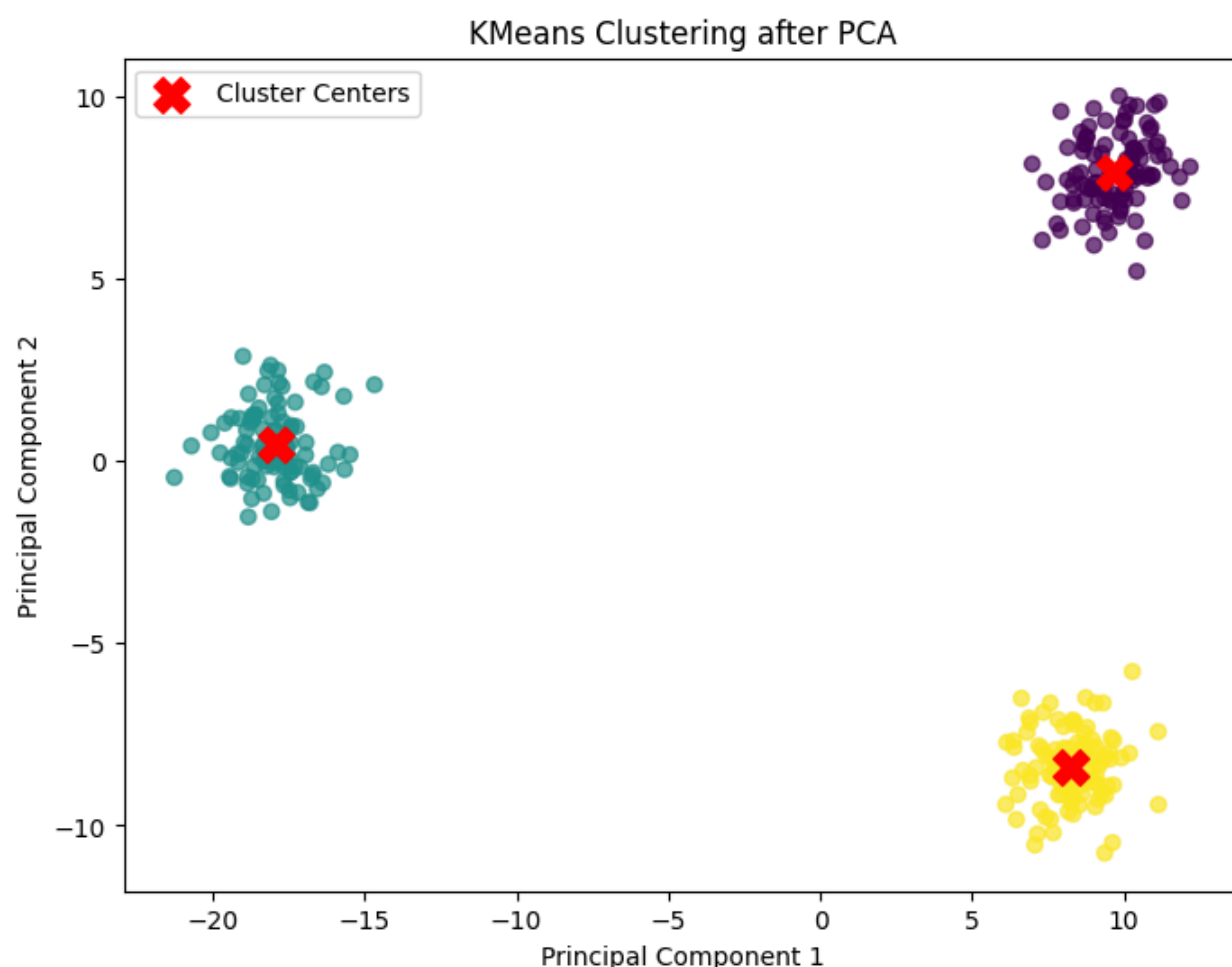
```
In [40]: from sklearn.datasets import make_blobs
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# تولید داده‌های مصنوعی با ابعاد بالا (۱۰ ویژگی) و ۳ خوشه
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0, n_features=10)

# برای کاهش ابعاد به ۲ مؤلفه اصلی (PCA) اعمال تحلیل مؤلفه‌های اصلی
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
# روی فضای کاهش‌یافته KMeans اعمال خوشه‌بندی
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X_pca)

# رسم نتایج خوشه‌بندی در فضای دو مؤلفه اصلی
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='X', s=200, label='Cluster Centers')
plt.title('KMeans Clustering after PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، ترکیب دو تکنیک قدرتمند یعنی تحلیل مؤلفه‌های اصلی (PCA) و خوشه‌بندی KMeans برای تحلیل داده‌های مصنوعی با ابعاد بالا به کار گرفته شده است:

- ابتدا داده‌های مصنوعی با ۱۰ ویژگی و ۳ خوشه تولید شده‌اند تا سناریوی خوشه‌بندی در فضای چندبعدی شبیه‌سازی شود.
- برای کاهش پیچیدگی و امکان تصویرسازی، از PCA برای کاهش ابعاد به ۲ مؤلفه اصلی استفاده شده است. این مؤلفه‌ها بیشترین واریانس داده را حفظ می‌کنند.
- سپس الگوریتم KMeans روی داده‌های کاهش‌یافته اعمال شده تا خوشه‌ها در فضای جدید شناسایی شوند.
- در پایان، نقاط داده با رنگ‌بندی بر اساس خوشه‌ها و مراکز خوشه با علامت قرمز رسم شده‌اند تا ساختار خوشه‌بندی به‌وضوح قابل مشاهده باشد.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از کاهش ابعاد، داده‌های پیچیده را ساده‌سازی کرد و سپس با خوشه‌بندی، الگوهای پنهان در آن‌ها را کشف نمود. این ترکیب در بسیاری از کاربردهای عملی مانند تحلیل رفتار مشتری، فشرده‌سازی داده و تصویرسازی داده‌های چندبعدی بسیار مؤثر است.

تجزیه ماتریس غیرمنفی (Nonnegative Matrix Factorization - NMF) +

تجزیه ماتریس غیرمنفی (Nonnegative Matrix Factorization - NMF) یک تکنیک کاهش ابعاد (dimensionality reduction) است که یک ماتریس داده غیرمنفی را به دو ماتریس با ابعاد پایین‌تر تجزیه می‌کند، که در آن تمام عناصر غیرمنفی هستند. یک ماتریس نشان‌دهنده بردارهای پایه (ویژگی‌ها) و دیگری نشان‌دهنده ضرایب برای هر نقطه داده بر اساس این بردارهای پایه است. NMF به ویژه زمانی مفید است که داده‌ها دارای مقادیر غیرمنفی باشند و زمانی که ویژگی‌های مبتنی بر بخش (parts-based) و قابل تفسیر مورد نظر هستند.

نکات کلیدی درباره NMF:

- محدودیت غیرمنفی بودن (Nonnegativity Constraint):** هم بردارهای پایه و هم ضرایب در NMF محدود به غیرمنفی بودن هستند، که باعث می‌شود فاکتورهای حاصل جمعی و قابل تفسیر باشند.
- کاربردها در خوشه‌بندی (Applications in Clustering):** NMF اغلب برای وظایف خوشه‌بندی (clustering tasks) به کار گرفته می‌شود، زیرا می‌تواند نمایش‌های مبتنی بر بخش از داده‌ها را آشکار کند و تفسیر ساختار زیربنایی را آسان‌تر سازد.
- مناسب برای داده‌های متنی و تصویری (Suitability for Text and Image Data):** NMF معمولاً در پردازش زبان طبیعی (natural language processing) برای مدل‌سازی موضوع (topic modeling) و در تحلیل تصویر (image analysis) برای استخراج مؤلفه‌های معنادار استفاده می‌شود.
- پارامترها (Parameters):** تعداد مؤلفه‌ها در تجزیه، یک پارامتر حیاتی است که ابعاد فضای کاهش یافته را تعیین می‌کند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای NMF اعمال شده بر خوشه‌بندی ارائه دهیم:

```
In [43]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import NMF
from sklearn.cluster import KMeans
from sklearn.pipeline import make_pipeline
import numpy as np
import re

# Newsgroups مرحله ۱: بارگذاری مجموعه داده متنی 20
# این مجموعه شامل هزاران سند متنی از ۲۰ گروه خبری مختلف است
newsgroups = fetch_20newsgroups(
    subset='all',
    remove=('headers', 'footers', 'quotes') # حذف اطلاعات متا برای تمرکز روی محتوای اصلی
)

# مرحله ۲: پیش‌پردازش متون
# تبدیل به حروف کوچک + حذف کاراکترهای غیرالفبایی
def preprocess(text):
    text = text.lower()
    text = re.sub(r'^a-z0-9\s', ' ', text)
    return text

cleaned_data = [preprocess(doc) for doc in newsgroups.data]

# مرحله ۳: تعریف پایپ‌لاین پردازش و خوشه‌بندی
# KMeans خوشه‌بندی با NMF استخراج موضوع با TF-IDF شامل سه مرحله: بردار
vectorizer = TfidfVectorizer(
    stop_words='english', # حذف کلمات رایج انگلیسی
    max_df=0.95, # حذف کلمات بسیار پرتکرار
    min_df=5 # حذف کلمات بسیار نادر
```

```
)

nmf = NMF(
    n_components=20,          # تعداد مؤلفه‌ها (موضوعات) برابر با تعداد خوشه‌ها
    random_state=42
)

kmeans = KMeans(
    n_clusters=20,           # تعداد خوشه‌ها برابر با تعداد گروه‌های خبری
    random_state=42
)

pipeline = make_pipeline(vectorizer, nmf, kmeans)

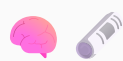
# مرحله ۴: آموزش پایپ‌لاین روی داده‌های متنی پیش‌پردازش‌شده
pipeline.fit(cleaned_data)

# NMF مرحله ۵: نمایش ۱۰ کلمه برتر برای هر موضوع استخراج‌شده توسط
feature_names = vectorizer.get_feature_names_out()
top_words = np.argsort(nmf.components_, axis=1)[: , :-11:-1] # انتخاب ۱۰ کلمه با بیشترین وزن

print("Top words per topic:\n")
for i, topic in enumerate(top_words):
    words = [feature_names[j] for j in topic]
    print(f"Topic {i + 1}: {' ', ' '.join(words)}")
```

Top words per topic:

Topic 1: don, just, think, want, say, know, ll, really, read, oh
Topic 2: window, problem, application, server, motif, display, manager, using, use, screen
Topic 3: god, jesus, bible, christ, believe, faith, christian, christians, sin, church
Topic 4: drive, disk, hard, drives, floppy, ide, boot, meg, cd, controller
Topic 5: game, games, team, year, hockey, baseball, season, play, players, espn
Topic 6: thanks, advance, hi, looking, help, info, mail, appreciated, information, hello
Topic 7: edu, mail, com, list, address, send, email, information, internet, university
Topic 8: geb, dsl, n3jxp, cadre, chastity, pitt, skepticism, intellect, shameful, surrender
Topic 9: key, chip, encryption, clipper, keys, escrow, government, algorithm, phone, use
Topic 10: armenian, armenians, turkish, genocide, armenia, turkey, muslim, turks, soviet, russian
Topic 11: card, monitor, video, vga, cards, color, drivers, bus, apple, ram
Topic 12: windows, dos, os, ms, nt, run, microsoft, running, version, apps
Topic 13: car, cars, bike, engine, new, miles, dealer, buy, oil, ford
Topic 14: does, know, anybody, mean, let, appreciated, work, doesn, program, info
Topic 15: 00, 10, 50, sale, 20, 15, shipping, 25, new, 11
Topic 16: file, files, program, image, format, ftp, gif, graphics, directory, zip
Topic 17: people, government, gun, right, fbi, law, did, koresh, batf, guns
Topic 18: israel, jews, israeli, arab, jewish, arabs, peace, palestinian, palestinians, state
Topic 19: scsi, ide, controller, bus, isa, data, pc, bit, dma, devices
Topic 20: like, good, ve, time, got, use, really, years, space, long



توضیحات مثال کد پایتون

در این مثال، یک پایپ‌لاین کامل برای تحلیل موضوعی و خوشه‌بندی متون خبری با استفاده از مجموعه داده معروف **Newsgroups 20** پیاده‌سازی شده است. این مجموعه شامل هزاران سند متنی از ۲۰ گروه خبری مختلف است:

- ابتدا داده‌های متنی بارگذاری شده و با حذف کاراکترهای غیرالفبایی و تبدیل به حروف کوچک، پیش‌پردازش شده‌اند.
- سپس با استفاده از **TF-IDF**، متون به بردارهای عددی تبدیل شده‌اند که وزن هر واژه را نسبت به اهمیت آن در سند مشخص می‌کند.
- با الگوریتم **NMF**، مؤلفه‌های اصلی (موضوعات) استخراج شده‌اند. این روش بر پایه تجزیه ماتریس و ساختار افزایشی عمل می‌کند و برای تحلیل موضوعی بسیار مناسب است.
- در مرحله آخر، با استفاده از **KMeans**، اسناد بر اساس نمایش موضوعی‌شان خوشه‌بندی شده‌اند.
- برای هر موضوع، ۱۰ واژه کلیدی با بیشترین وزن نمایش داده شده‌اند که نمایانگر محتوای غالب آن موضوع هستند.

این مثال نشان می‌دهد که چگونه می‌توان با ترکیب **بردارسازی متنی، تحلیل مؤلفه‌ها و خوشه‌بندی**، ساختار پنهان در مجموعه‌های بزرگ متنی را کشف کرد. این روش در کاربردهایی مانند دسته‌بندی خودکار اسناد، تحلیل احساسات، و توصیه‌گرهای متنی بسیار مؤثر است.

خوشه‌بندی طیفی (Spectral Clustering)

خوشه‌بندی طیفی (Spectral Clustering) تکنیکی است که برای انجام خوشه‌بندی (clustering) از ویژگی‌های طیفی داده‌ها بهره می‌برد. این روش به ویژه هنگام کار با خوشه‌های غیرمحدب (non-convex) و ساختارهای هندسی پیچیده مؤثر است. خوشه‌بندی طیفی شامل گام‌های زیر است:

1. **ساخت گراف (Graph Construction):** ساخت یک ماتریس وابستگی (affinity matrix) که شباهت دو به دوی بین نقاط داده را نمایش می‌دهد. معیارهای رایج وابستگی شامل شباهت گوسی (Gaussian similarity) یا نزدیک‌ترین همسایه‌ها (nearest neighbors) هستند.
2. **تبدیل گراف (Graph Transformation):** تبدیل ماتریس وابستگی به یک ماتریس لاپلاسی گراف (graph Laplacian matrix). ماتریس لاپلاسی اتصالات و روابط بین نقاط داده را ثبت می‌کند.
3. **تجزیه مقادیر ویژه (Eigenvalue Decomposition):** محاسبه مقادیر ویژه (eigenvalues) و بردارهای ویژه (eigenvectors) ماتریس لاپلاسی. بردارهای ویژه مربوط به کوچک‌ترین مقادیر ویژه، ساختار خوشه را در داده‌ها رمزگذاری می‌کنند.
4. **کاهش ابعاد (Dimensionality Reduction):** استفاده از بردارهای ویژه مربوط به کوچک‌ترین مقادیر ویژه به عنوان یک نمایش کاهش‌یافته از داده‌ها.
5. **خوشه‌بندی (Clustering):** اعمال یک الگوریتم خوشه‌بندی استاندارد (مانند KMeans) بر روی نمایش کاهش‌یافته برای به دست آوردن خوشه‌های نهایی.

این روش برای شناسایی خوشه‌هایی با شکل‌های پیچیده قدرتمند است و در برابر نویز و داده‌های پرت (outliers) مقاوم می‌باشد.

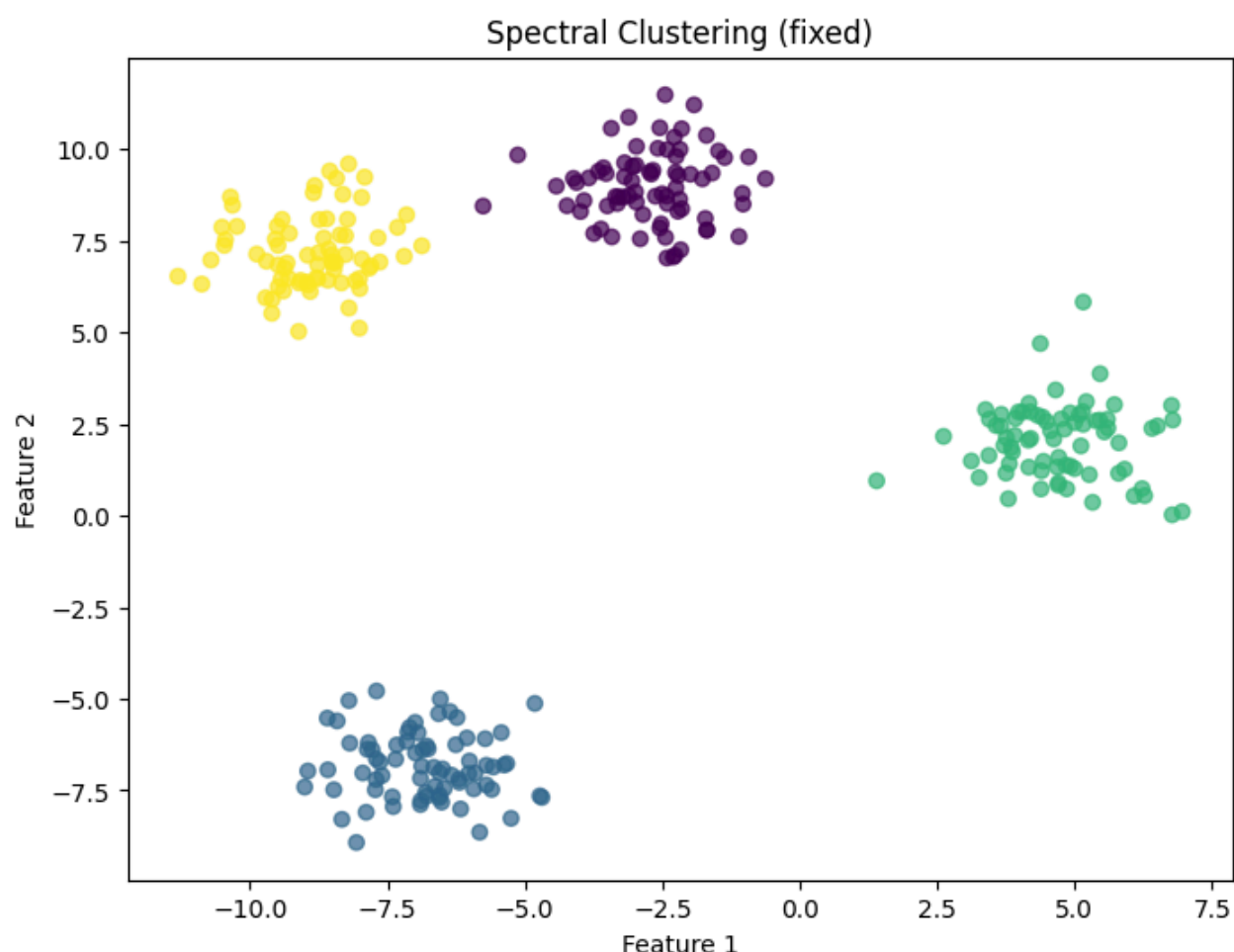
مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی طیفی:

```
In [50]: from sklearn.datasets import make_blobs
from sklearn.cluster import SpectralClustering
import matplotlib.pyplot as plt

# تولید داده‌های مصنوعی با ۴ خوشه و پراکندگی مشخص
X, _ = make_blobs(n_samples=300, centers=4, random_state=42, cluster_std=1.0)

# Nearest Neighbors از نوع affinity اعمال خوشه‌بندی طیفی با
spectral = SpectralClustering(
    n_clusters=4,
    affinity='nearest_neighbors',
    n_neighbors=120, # اندکی بیاندیشید و بحث کنید
    random_state=42
)
labels = spectral.fit_predict(X)

# رسم نتایج خوشه‌بندی
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Spectral Clustering (fixed)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، الگوریتم خوشه‌بندی طیفی (Spectral Clustering) با استفاده از ساختار گراف همسایگی روی داده‌های مصنوعی اعمال شده است. این روش برای شناسایی خوشه‌های پیچیده و غیرخطی بسیار مؤثر است:

- ابتدا داده‌هایی با ۴ خوشه و ۳۰۰ نمونه تولید شده‌اند تا ساختار خوشه‌ای قابل تحلیل باشد.
- برای ساخت گراف همسایگی، از پارامتر `n_neighbors=120` استفاده شده است. این انتخاب نقش کلیدی در کیفیت خوشه‌بندی دارد.
- در الگوریتم Spectral Clustering، گراف همسایگی باید **connected** باشد تا محاسبه eigenvectors برای embedding داده‌ها به‌درستی انجام شود.
- اگر مقدار `n_neighbors` خیلی کم باشد (مثلاً زیر 30)، احتمال دارد گراف disconnect شود و الگوریتم با هشدار یا نتایج ضعیف مواجه شود.
- اگر مقدار خیلی زیاد باشد (مثلاً نزدیک به تعداد کل نمونه‌ها)، گراف بیش از حد متصل شده و ساختار محلی داده‌ها از بین می‌رود.
- انتخاب ۱۲۰ برای ۳۰۰ نمونه، یک تعادل مناسب بین اتصال گراف و حفظ ساختار محلی ایجاد می‌کند. این مقدار تقریباً بین $\log(n)$ و $\sqrt{n}$ ضربدر یک ضریب ایمنی است.

این مثال نشان می‌دهد که در خوشه‌بندی طیفی، انتخاب مناسب `n_neighbors` نه تنها از نظر عددی بلکه از نظر ساختار گرافی اهمیت بالایی دارد. در پروژه‌های واقعی، این پارامتر باید با آزمون و خطا یا تحلیل ساختار داده تنظیم شود.

Section 9.5

9.5.2. Similarity measures

معیارهای شباهت (Similarity Measures)

در زمینه خوشه‌بندی داده‌های گراف و شبکه، معیارهای شباهت (similarity measures) نقش حیاتی در کمی‌سازی میزان شباهت یا ارتباط بین گره‌ها ایفا می‌کنند. این معیارها به ساخت ماتریس‌های وابستگی (affinity matrices) کمک می‌کنند که برای الگوریتم‌های خوشه‌بندی مختلف ضروری هستند. برخی از معیارهای رایج شباهت برای داده‌های گراف و شبکه عبارتند از:

- **Jaccard Similarity (شباهت ژاکارد):** شباهت بین دو مجموعه را با تقسیم اندازه اشتراک آن‌ها بر اندازه اجتماع آن‌ها اندازه‌گیری می‌کند.
- **Cosine Similarity (شباهت کسینوسی):** کسینوس زاویه بین دو بردار را اندازه‌گیری می‌کند. در زمینه گراف‌ها، بردارها نشان‌دهنده همسایگی گره (node neighborhoods) یا بردارهای ویژگی (attribute vectors) هستند.
- **Graph Edit Distance (فاصله ویرایش گراف):** با اندازه‌گیری حداقل هزینه تبدیل یک گراف به گراف دیگر از طریق یک سری عملیات ویرایشی (مانند حذف، درج گره یا اصلاح یال)، عدم شباهت بین دو گراف را کمی‌سازی می‌کند.
- **Node and Edge Overlap (همپوشانی گره و یال):** همپوشانی گره‌ها یا یال‌ها بین دو گراف را اندازه‌گیری می‌کند و یک معیار ساده از شباهت بر اساس عناصر مشترک را فراهم می‌سازد.
- **Pearson Correlation Coefficient (ضریب همبستگی پیرسون):** همبستگی خطی (linear correlation) بین دو متغیر را اندازه‌گیری می‌کند. در زمینه گراف‌ها، می‌توان از آن برای اندازه‌گیری همبستگی بین ویژگی‌های گره (node attributes) استفاده کرد.

انتخاب یک معیار شباهت مناسب به ویژگی‌های داده‌های گراف و وظیفه خوشه‌بندی خاص بستگی دارد.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای محاسبه Jaccard Similarity بر روی داده‌های گراف ارائه دهیم:

```
In [53]: import networkx as nx
from sklearn.metrics import jaccard_score
import numpy as np

# ایجاد دو گراف نمونه برای مقایسه
G1 = nx.Graph([(1, 2), (2, 3), (3, 4)])
G2 = nx.Graph([(1, 2), (2, 3), (3, 4), (4, 5)])

# تبدیل گراف‌ها به ماتریس مجاورت (Adjacency Matrix)
adj_matrix1 = nx.to_numpy_array(G1)
adj_matrix2 = nx.to_numpy_array(G2)

# صفر padding یکسان‌سازی ابعاد ماتریس‌ها با
max_size = max(adj_matrix1.shape[0], adj_matrix2.shape[0])
if adj_matrix1.shape[0] < max_size:
    adj_matrix1 = np.pad(adj_matrix1, ((0, max_size - adj_matrix1.shape[0]), (0, max_size - adj_matrix1.shape[1])))
if adj_matrix2.shape[0] < max_size:
    adj_matrix2 = np.pad(adj_matrix2, ((0, max_size - adj_matrix2.shape[0]), (0, max_size - adj_matrix2.shape[1])))

# تبدیل ماتریس‌ها به بردارهای یک‌بعدی برای محاسبه شباهت
flat_matrix1 = adj_matrix1.flatten()
flat_matrix2 = adj_matrix2.flatten()

# محاسبه شباهت جاکارد بین دو گراف
jaccard_similarity = jaccard_score(flat_matrix1, flat_matrix2)

# نمایش نتیجه
print(f"Jaccard Similarity between G1 and G2: {jaccard_similarity}")
```

Jaccard Similarity between G1 and G2: 0.75

توضیحات مثال کد پایتون

در این مثال، هدف مقایسه ساختار دو گراف با استفاده از شباهت جاکارد (Jaccard Similarity) است. این معیار نشان می‌دهد که تا چه حد یال‌های دو گراف با یکدیگر همپوشانی دارند:

- ابتدا دو گراف ساده با تعداد متفاوتی از یال‌ها ایجاد شده‌اند. گراف دوم یک یال بیشتر دارد.
- هر گراف به **ماتریس مجاورت** تبدیل شده که نشان‌دهنده ارتباط بین گره‌هاست.
- برای مقایسه، ابعاد ماتریس‌ها با padding صفر یکسان‌سازی شده‌اند تا بتوان آن‌ها را به بردارهای یک‌بعدی تبدیل کرد.
- سپس با استفاده از **jaccard_score**، میزان شباهت بین دو بردار محاسبه شده است. این مقدار بین ۰ تا ۱ است و هرچه بیشتر باشد، شباهت ساختاری بین گراف‌ها بیشتر است.

این روش برای مقایسه گراف‌ها در کاربردهایی مانند تحلیل شبکه‌های اجتماعی، تشخیص تغییرات ساختاری در گراف‌ها، و ارزیابی الگوریتم‌های تولید گراف بسیار مفید است. البته برای گراف‌های بزرگ‌تر و پیچیده‌تر، معیارهای پیشرفته‌تری مانند Graph Edit Distance یا Graph Kernels نیز استفاده می‌شود.

9.5.3. Graph clustering methods

روش‌های خوشه‌بندی گراف (Graph Clustering Methods)

روش‌های خوشه‌بندی گراف (Graph clustering methods) با هدف شناسایی گروه‌هایی از گره‌ها در داخل یک گراف که به طور چگال‌تری به یکدیگر متصل هستند تا به گره‌های خارج از گروه، عمل می‌کنند. خوشه‌بندی در گراف‌ها به ویژه برای کشف ساختارهای جامعه (community structures)، شناسایی ماژول‌های کارکردی (functional modules) یا تشخیص گروه‌های منسجم از موجودیت‌های مرتبط مفید است. برخی از روش‌های رایج خوشه‌بندی گراف عبارتند از:

1. **Louvain Modularity Optimization**: مدیولاریتی (modularity) یک پارتیشن را حداکثر می‌کند، که چگالی یال‌ها را در داخل خوشه‌ها در مقایسه با چگالی مورد انتظار در یک شبکه تصادفی اندازه‌گیری می‌کند.
2. **Spectral Clustering (خوشه‌بندی طیفی)**: تجزیه طیفی (spectral decomposition) را بر روی ماتریس لاپلاسی گراف (Laplacian matrix) اعمال می‌کند تا خوشه‌ها را بر اساس بردارهای ویژه (eigenvectors) مربوط به کوچک‌ترین مقادیر ویژه شناسایی کند.
3. **Walktrap**: شباهت بین گره‌ها را بر اساس پیمایش تصادفی (random walks) در گراف اندازه‌گیری می‌کند، به طوری که گره‌هایی که اغلب با هم پیمایش می‌شوند، مشابه‌تر در نظر گرفته می‌شوند.
4. **Infomap**: شبکه را به عنوان یک جریان اطلاعات مدل‌سازی کرده و به دنبال یافتن پارتیشن‌هایی است که طول توصیف (description length) جریان اطلاعات را فشرده می‌کند.
5. **Kernighan-Lin Bisection**: یک روش اکتشافی (heuristic method) است که به صورت بازگشتی گراف را به دو نیمه تقسیم می‌کند و یک تابع هزینه مربوط به برش یال (edge cuts) را بهینه می‌سازد.

این روش‌ها می‌توانند بر روی انواع مختلفی از گراف‌ها، از جمله شبکه‌های اجتماعی، شبکه‌های بیولوژیکی، شبکه‌های استنادی (citation networks) و غیره اعمال شوند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی گراف با استفاده از Louvain Modularity Optimization ارائه دهیم:

```
In [ ]: pip install community
```

```
In [ ]: pip install python-louvain
```

```
In [2]: import networkx as nx
import community.community_louvain as community_louvain # برای خوشه‌بندی گراف Louvain کتابخانه
import matplotlib.pyplot as plt

# Karate Club Graph (ایجاد گراف معروف باشگاه کاراته)
# این گراف شامل ۳۴ گره و روابط اجتماعی بین اعضای باشگاه است
G = nx.karate_club_graph()

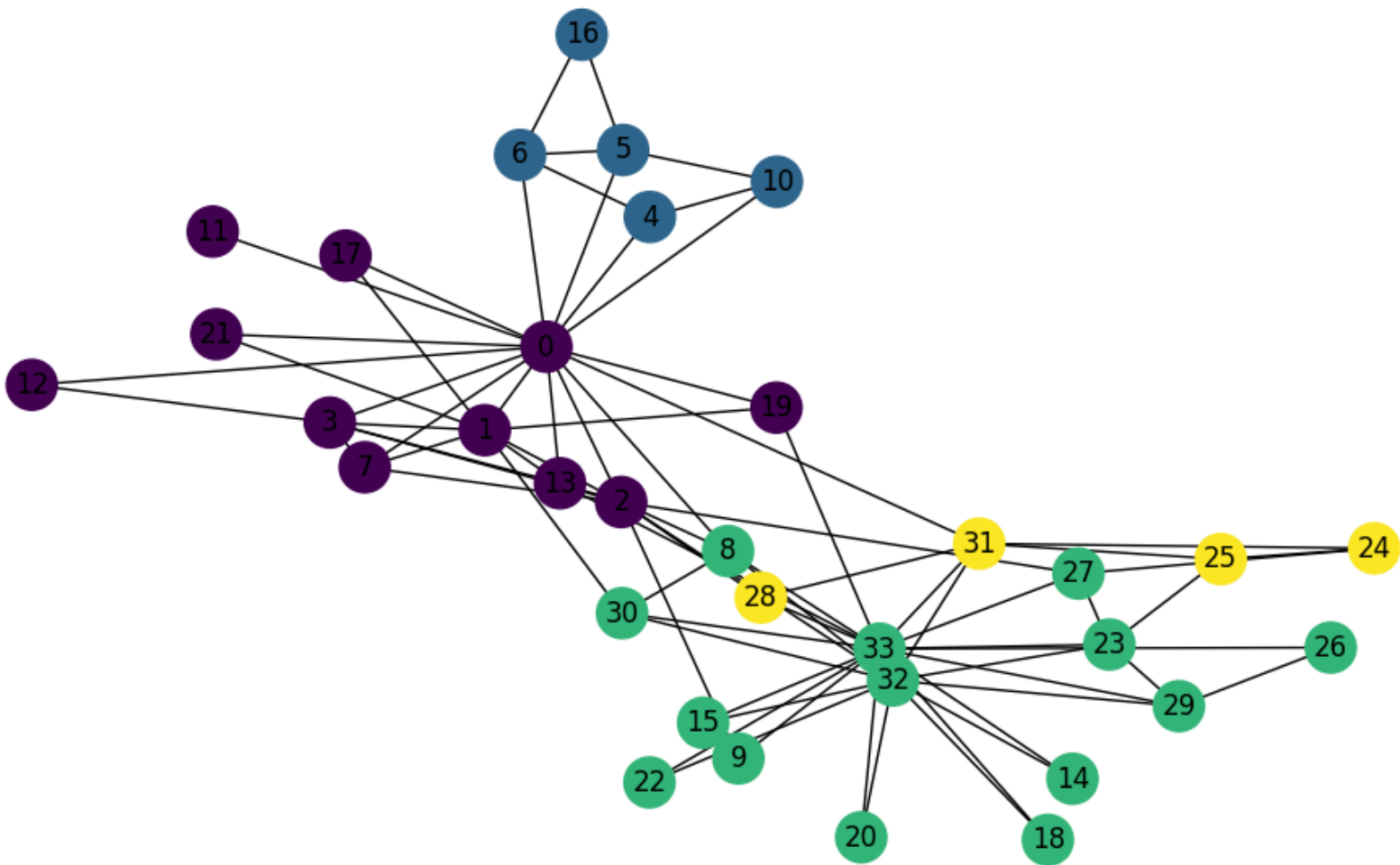
# برای خوشه‌بندی گراف بر اساس ماژولاریتی Louvain اعمال الگوریتم
partition = community_louvain.best_partition(G)

# فنری Layout تعیین موقعیت گره‌ها برای رسم گراف با
pos = nx.spring_layout(G, seed=42) # برای تکرارپذیری نتایج

# تعیین رنگ هر گره بر اساس خوشه‌ای که به آن تعلق دارد
colors = [partition[node] for node in G.nodes]

# رسم گراف با رنگ‌بندی خوشه‌ها
plt.figure(figsize=(10, 6))
nx.draw(G, pos, node_color=colors, with_labels=True, cmap=plt.cm.viridis, node_size=500)
plt.title('Graph Clustering using Louvain Modularity Optimization')
plt.show()
```

Graph Clustering using Louvain Modularity Optimization



توضیحات مثال کد پایتون

- در این مثال، الگوریتم **Louvain Modularity Optimization** برای خوشه‌بندی گراف اجتماعی معروف **باشگاه کاراته** استفاده شده است. این الگوریتم یکی از روش‌های محبوب برای کشف جوامع در شبکه‌های پیچیده است:
- گراف باشگاه کاراته شامل ۳۴ گره است که روابط اجتماعی بین اعضای باشگاه را مدل‌سازی می‌کند. این گراف در مطالعات شبکه‌های اجتماعی بسیار معروف است.
 - الگوریتم Louvain با هدف **بیشینه‌سازی ماژولاریتی**، گره‌ها را به خوشه‌هایی تقسیم می‌کند که درون خوشه‌ای ارتباط بیشتری نسبت به بین‌خوشه‌ای دارند.
 - با استفاده از `best_partition`، برچسب خوشه برای هر گره تعیین شده و سپس گراف با رنگ‌بندی خوشه‌ها رسم شده است.
 - برای تکرارپذیری نتایج، از `spring_layout` با مقدار ثابت `seed` استفاده شده تا موقعیت گره‌ها در هر اجرا مشابه باشد.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی مبتنی بر ماژولاریتی**، ساختارهای پنهان در شبکه‌های اجتماعی را کشف کرد. الگوریتم Louvain به دلیل سرعت بالا و دقت مناسب، در تحلیل شبکه‌های بزرگ و پیچیده بسیار پرکاربرد است.

Section 9.6

9.6.1. Semisupervised clustering on partially labeled data

خوشه‌بندی نیمه‌نظارتی (Semi-supervised clustering)

خوشه‌بندی نیمه‌نظارتی (Semi-supervised clustering) به سناریوهایی می‌پردازد که در آن‌ها تنها زیرمجموعه‌ای از نقاط داده در یک مجموعه داده دارای برچسب هستند. این رویکرد داده‌های دارای برچسب و بدون برچسب را ترکیب می‌کند تا فرآیند خوشه‌بندی را بهبود بخشد. در داده‌های با برچسب جزئی (partially labeled data)، تنها بخشی از نقاط داده دارای برچسب‌های واقعیت زمینی (ground truth labels) هستند، در حالی که بقیه بدون برچسب می‌باشند. روش‌های خوشه‌بندی نیمه‌نظارتی با هدف بهره‌برداری از اطلاعات دارای برچسب موجود برای هدایت خوشه‌بندی کل مجموعه داده عمل می‌کنند.

نکات کلیدی درباره خوشه‌بندی نیمه‌نظارتی بر روی داده‌های با برچسب جزئی:

- **ترکیب اطلاعات با برچسب (Incorporating Labeled Information):** خوشه‌بندی نیمه‌نظارتی نمونه‌های دارای برچسب را در فرآیند خوشه‌بندی ادغام می‌کند و دقت انتساب خوشه‌ها را افزایش می‌دهد.
- **محدودیت‌های نرم (Soft Constraints):** الگوریتم‌ها اغلب از محدودیت‌های نرمی استفاده می‌کنند که رفتار مشابهی را بین نقاط داده با برچسب و بدون برچسب تشویق می‌کند و به مدل اجازه می‌دهد از هر دو منبع یاد بگیرد.
- **رویکردهای ترکیبی (Hybrid Approaches):** روش‌های خوشه‌بندی نیمه‌نظارتی می‌توانند تکنیک‌های خوشه‌بندی سنتی را با الگوریتم‌های یادگیری نظارتی (supervised learning algorithms) ترکیب کنند تا یک مدل ترکیبی ایجاد شود.
- **رسیدگی به برچسب‌های نویزی (Handling Noisy Labels):** این روش‌ها برای رسیدگی به نویز و خطاها در داده‌های دارای برچسب طراحی شده‌اند و خوشه‌بندی قوی را در حضور برچسب‌های ناقص فراهم می‌کنند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی نیمه‌نظارتی بر روی داده‌های با برچسب جزئی ارائه دهیم:

```
In [5]: from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.semi_supervised import LabelPropagation
import numpy as np
import matplotlib.pyplot as plt

# تولید داده‌های مصنوعی با ۳ خوشه مشخص
X, y = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# شبیه‌سازی داده‌های نیمه‌برچسب‌خورده با حذف تصادفی ۹۰٪ از برچسب‌ها
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y)) < 0.9
y[random_unlabeled_points] = -1 # مقدار 1- نشان‌دهنده داده بدون برچسب است

# روی کل داده‌ها (بدون توجه به برچسب‌ها) KMeans اعمال خوشه‌بندی
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)

# با استفاده از داده‌های نیمه‌برچسب‌خورده Label Propagation اعمال الگوریتم
label_propagation = LabelPropagation(kernel='knn', n_neighbors=10)
label_propagation.fit(X, y)
```

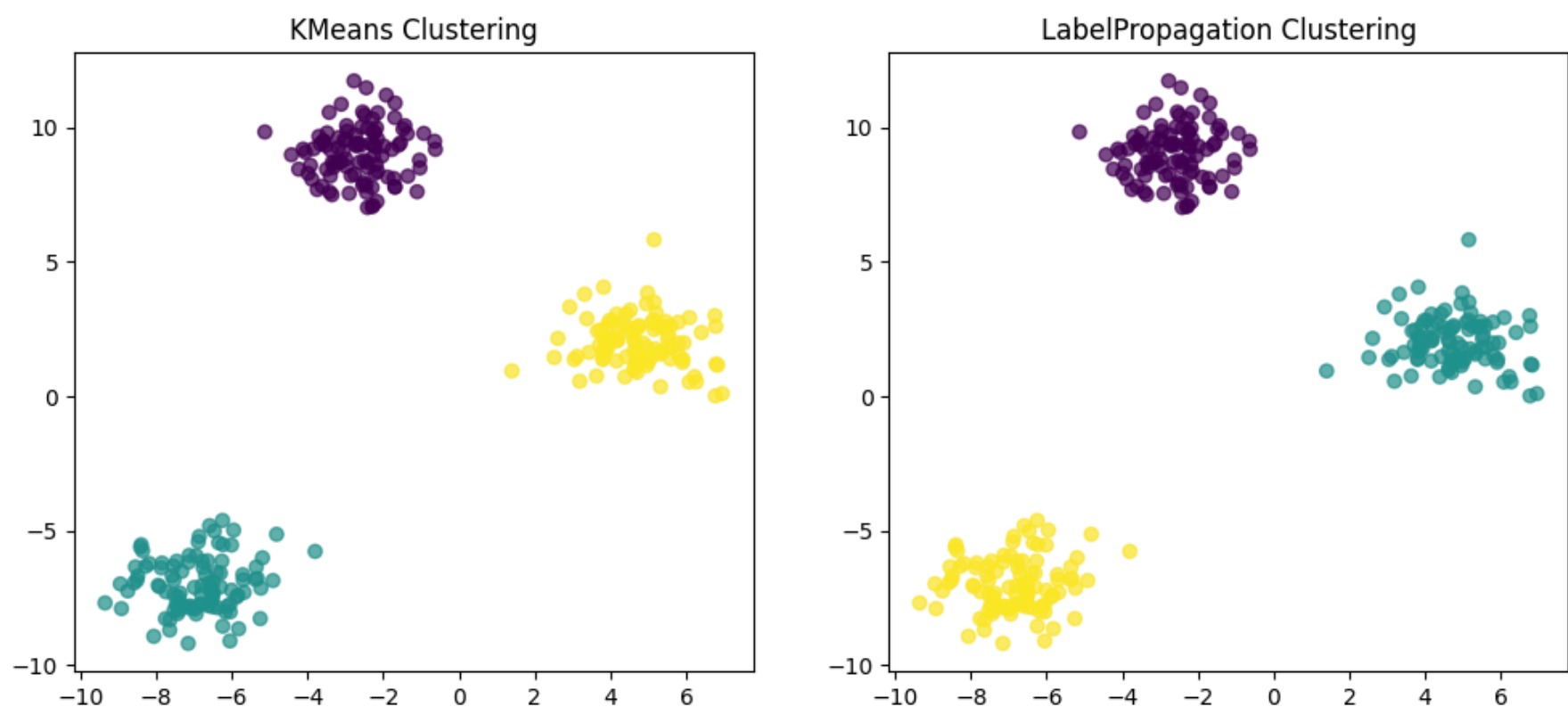
```
lp_labels = label_propagation.predict(X)

# رسم نتایج خوشه‌بندی برای هر الگوریتم
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis', alpha=0.7)
plt.title('KMeans Clustering')

plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=lp_labels, cmap='viridis', alpha=0.7)
plt.title('LabelPropagation Clustering')

plt.show()
```



توضیحات مثال کد پایتون

در این مثال، دو الگوریتم متفاوت برای خوشه‌بندی داده‌های مصنوعی با سه خوشه به کار گرفته شده‌اند: **KMeans** و **Label Propagation**. هدف، مقایسه عملکرد خوشه‌بندی سنتی با یادگیری نیمه‌نظارتی است:

- ابتدا داده‌هایی با سه خوشه تولید شده‌اند که ساختار مشخصی دارند.
- برای شبیه‌سازی یادگیری نیمه‌نظارتی، فقط ۱۰٪ از داده‌ها دارای برچسب نگه داشته شده‌اند و باقی با مقدار -1 به عنوان داده‌های بدون برچسب علامت‌گذاری شده‌اند.
- الگوریتم **KMeans** بدون توجه به برچسب‌ها، داده‌ها را خوشه‌بندی کرده و ساختار کلی را بر اساس فاصله‌ها شناسایی می‌کند.
- در مقابل، الگوریتم **Label Propagation** با استفاده از داده‌های برچسب‌خورده، برچسب‌ها را به داده‌های بدون برچسب منتقل می‌کند. این روش از ساختار گرافی داده‌ها و همسایگی برای انتشار اطلاعات استفاده می‌کند.
- نتایج هر دو الگوریتم به صورت تصویری نمایش داده شده‌اند تا تفاوت در نحوه خوشه‌بندی قابل مشاهده باشد.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **یادگیری نیمه‌نظارتی**، از داده‌های ناقص نیز برای ساخت مدل‌های طبقه‌بندی یا خوشه‌بندی استفاده کرد. همچنین مقایسه با KMeans کمک می‌کند تا مزایا و محدودیت‌های هر روش بهتر درک شوند.

خوشه‌بندی نیمه‌نظارتی با استفاده از محدودیت‌های زوجی (Pairwise constraints)

خوشه‌بندی نیمه‌نظارتی با استفاده از محدودیت‌های زوجی (pairwise constraints) شامل ارائه روابط یا محدودیت‌های زوجی بین نقاط داده به الگوریتم است. این محدودیت‌ها می‌توانند به شکل محدودیت‌های must-link (نشان‌دهنده اینکه دو نقطه باید به یک خوشه تعلق داشته باشند) یا محدودیت‌های cannot-link (نشان‌دهنده اینکه دو نقطه نمی‌توانند به یک خوشه تعلق داشته باشند) باشند. با ترکیب این اطلاعات زوجی، الگوریتم خوشه‌بندی گروه‌بندی نقاط داده را با در نظر گرفتن روابط مشخص شده، هدایت می‌کند.

نکات کلیدی درباره خوشه‌بندی نیمه‌نظارتی بر روی محدودیت‌های زوجی:

- **محدودیت‌های Must-Link و Cannot-Link:** محدودیت‌های must-link مشخص می‌کنند که دو نقطه داده باید به یک خوشه تعلق داشته باشند، در حالی که محدودیت‌های cannot-link مشخص می‌کنند که آن‌ها باید در خوشه‌های متفاوتی باشند.
- **ترکیب محدودیت:** الگوریتم‌ها از این محدودیت‌های زوجی برای هدایت فرآیند خوشه‌بندی بهره می‌برند و انتساب‌های خوشه را بر اساس روابط مشخص شده تنظیم می‌کنند.
- **رسیدگی به محدودیت‌های نویزی:** روش‌های خوشه‌بندی نیمه‌نظارتی بر روی محدودیت‌های زوجی برای رسیدگی به محدودیت‌های نویزی یا متناقض طراحی شده‌اند و مقاومت در برابر اطلاعات ناقص را فراهم می‌کنند.
- **بهبود دقت خوشه‌بندی:** با ترکیب محدودیت‌های زوجی، الگوریتم با هدف بهبود دقت انتساب‌های خوشه، به ویژه هنگام کار با ساختارهای پیچیده یا همپوشان، عمل می‌کند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی نیمه‌نظارتی بر روی محدودیت‌های زوجی ارائه دهیم:

```
In [ ]: pip install metric-learn
```

```
In [12]: from sklearn.datasets import make_blobs
from metric_learn import MMC_Supervised
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# مرحله ۱: تولید داده‌های مصنوعی با ۳ خوشه مشخص
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# مرحله ۲: یادگیری متریک با استفاده از الگوریتم نیمه‌نظارتی
# را به صورت ضمنی ایجاد می‌کند cannot-link و must-link این الگوریتم با استفاده از برجسب‌ها، محدودیت‌های
mmc = MMC_Supervised()
mmc.fit(X, y_true)
X_transformed = mmc.transform(X) # تبدیل داده‌ها به فضای جدید با متریک آموخته‌شده

# برای تصویرسازی PCA مرحله ۳: بررسی داده‌های تبدیل‌شده با کاهش بعد
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_transformed)

plt.figure(figsize=(6, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_true, cmap='viridis', alpha=0.7)
plt.title('PCA of Transformed Data (after MMC)')
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.show()

# روی داده‌های تبدیل‌شده KMeans مرحله ۴: اعمال خوشه‌بندی
kmeans = KMeans(n_clusters=3, random_state=42)
labels_kmeans = kmeans.fit_predict(X_transformed)

# مرحله ۵: رسم نتایج خوشه‌بندی قبل و بعد از یادگیری متریک
plt.figure(figsize=(12, 5))

# نمایش خوشه‌های واقعی
plt.subplot(1, 2, 1)
```


- ابتدا داده‌هایی با ۳ خوشه تولید شده‌اند که ساختار مشخصی دارند.
- الگوریتم **MMC_Supervised** با استفاده از برچسب‌های واقعی، محدودیت‌های **must-link** و **cannot-link** را به صورت ضمنی ایجاد کرده و متریک فاصله‌ای جدیدی را یاد می‌گیرد.
- سپس داده‌ها به فضای جدیدی تبدیل می‌شوند که در آن فاصله‌ها بر اساس متریک آموخته‌شده تنظیم شده‌اند.
- برای بررسی ساختار جدید داده‌ها، از **PCA** برای کاهش بعد و تصویرسازی استفاده شده است.
- در این فضای جدید، الگوریتم **KMeans** اعمال شده تا خوشه‌بندی بهینه‌تری انجام شود.
- نتایج خوشه‌بندی قبل و بعد از یادگیری متریک به صورت تصویری نمایش داده شده‌اند تا تأثیر MMC به وضوح قابل مشاهده باشد.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **یادگیری متریک نیمه‌نظارتی**، ساختار داده‌ها را بهبود داد و خوشه‌بندی دقیق‌تری انجام داد. این روش در کاربردهایی مانند تشخیص چهره، تحلیل رفتار و داده‌های پزشکی بسیار مؤثر است.

9.6.3. Other types of background knowledge for semisupervised clustering

استفاده از دانش پس‌زمینه در خوشه‌بندی نیمه‌نظارتی (Semi-supervised clustering with background knowledge)

در خوشه‌بندی نیمه‌نظارتی (semi-supervised clustering)، بهره‌برداری از دانش پس‌زمینه اضافی فراتر از محدودیت‌های زوجی (pairwise constraints) یا نمونه‌های دارای برچسب، می‌تواند فرآیند خوشه‌بندی را بیشتر بهبود بخشد. انواع دیگر دانش پس‌زمینه ممکن است شامل موارد زیر باشد:

- **محدودیت‌های ویژگی (Attribute Constraints):** محدودیت‌ها بر روی ویژگی‌ها یا صفات نقاط داده می‌توانند الگوریتم خوشه‌بندی را هدایت کنند. به عنوان مثال، مشخص کردن اینکه ویژگی‌های خاصی باید در میان خوشه‌ها مشابه یا نامشابه باشند.
- **اطلاعات سلسله‌مراتبی (Hierarchy Information):** اطلاعات مربوط به روابط سلسله‌مراتبی بین خوشه‌ها یا نقاط داده می‌تواند ترکیب شود. این مورد به ویژه زمانی مفید است که داده‌ها ساختار تودرتو یا سلسله‌مراتبی از خود نشان می‌دهند.
- **احتمالات پیشین (Prior Probabilities):** ارائه احتمالات پیشین برای تعلق نقاط داده به خوشه‌های خاص می‌تواند الگوریتم را در شرایطی که انتظار می‌رود خوشه‌های خاصی غالب‌تر باشند، هدایت کند.
- **محدودیت‌های فضایی (Spatial Constraints):** ترکیب محدودیت‌های مبتنی بر آرایش فضایی (spatial arrangement) نقاط داده می‌تواند مفید باشد، به ویژه در کاربردهایی که در آن‌ها نزدیکی یا آرایش حاوی اطلاعات معنادار است.
- **اطلاعات زمانی (Temporal Information):** برای داده‌های سری زمانی، محدودیت‌های زمانی یا اطلاعات مربوط به ترتیب زمانی نقاط داده می‌تواند در هدایت فرآیند خوشه‌بندی ارزشمند باشد.

ترکیب انواع متنوع دانش پس‌زمینه به الگوریتم‌های خوشه‌بندی نیمه‌نظارتی اجازه می‌دهد تا با ویژگی‌های داده‌های مختلف و الزامات خاص کاربردها سازگار شوند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای خوشه‌بندی نیمه‌نظارتی با محدودیت‌های ویژگی ارائه دهیم:

```
In [23]: from sklearn.datasets import make_blobs
from sklearn.semi_supervised import LabelSpreading
from sklearn.cluster import SpectralClustering
import numpy as np
import matplotlib.pyplot as plt

# مرحله ۱: تولید داده‌های مصنوعی چالش‌برانگیز با همپوشانی بالا
X, y = make_blobs(n_samples=300, centers=3, cluster_std=2.5, random_state=42)

# اطلاعات ویژگی (برای آموزش یا نمایش، در الگوریتم استفاده نمی‌شود)
attribute_information = np.random.rand(len(y), 5)

# شبیه‌سازی محدودیت‌های ویژگی بین خوشه‌ها (برای اهداف آموزشی)
attribute_constraints = []
for i in range(5):
    cluster1, cluster2 = np.random.choice(range(3), size=2, replace=False)
    attribute_constraints.append((cluster1, cluster2, i))

# مرحله ۲: شبیه‌سازی یادگیری نیمه‌نظارتی با حذف ۹۰٪ از برچسب‌ها
y_partial = np.copy(y)
rng = np.random.RandomState(42)
for cls in np.unique(y):
    cls_indices = np.where(y == cls)[0]
    unlabeled_count = int(0.9 * len(cls_indices))
    unlabeled_indices = rng.choice(cls_indices, size=unlabeled_count, replace=False)
    y_partial[unlabeled_indices] = -1 # مقدار 1- نشان‌دهنده داده بدون برچسب است

# مرحله ۳: Spectral Clustering خوشه‌بندی بدون استفاده از برچسب‌ها با
spectral = SpectralClustering(n_clusters=3, random_state=42)
spectral_labels = spectral.fit_predict(X)

# مرحله ۴: LabelSpreading خوشه‌بندی نیمه‌نظارتی با استفاده از
label_spreading = LabelSpreading(kernel='knn', n_neighbors=10, alpha=0.8)
label_spreading.fit(X, y_partial)
lp_labels = label_spreading.transduction_

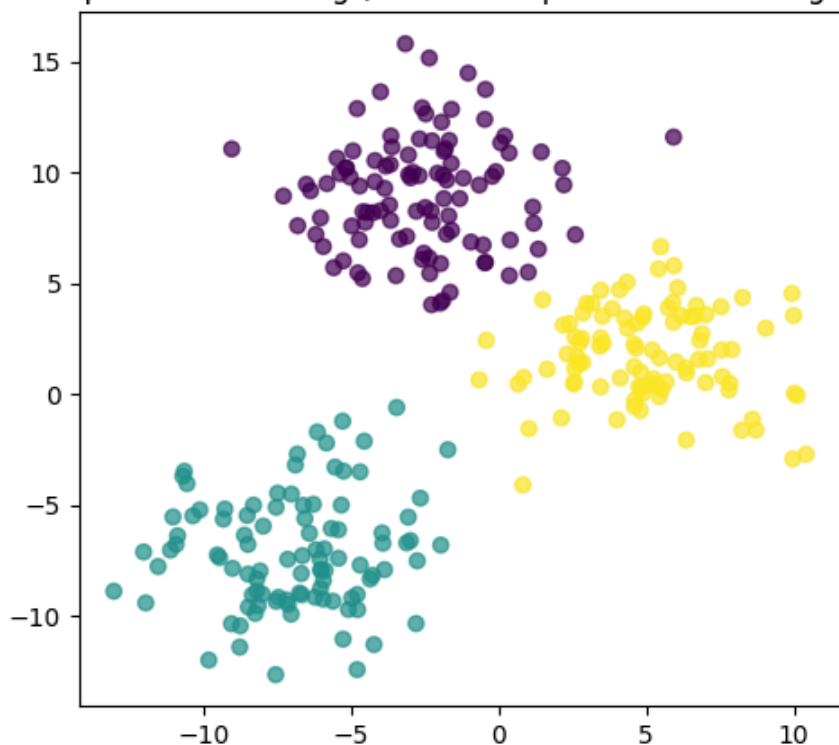
# مرحله ۵: رسم نتایج خوشه‌بندی
plt.figure(figsize=(12, 5))

# نمودار اول: خوشه‌بندی طیفی بدون دانش نیمه‌نظارتی
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=spectral_labels, cmap='viridis', alpha=0.7)
plt.title('Spectral Clustering (No Semi-supervised Knowledge)')

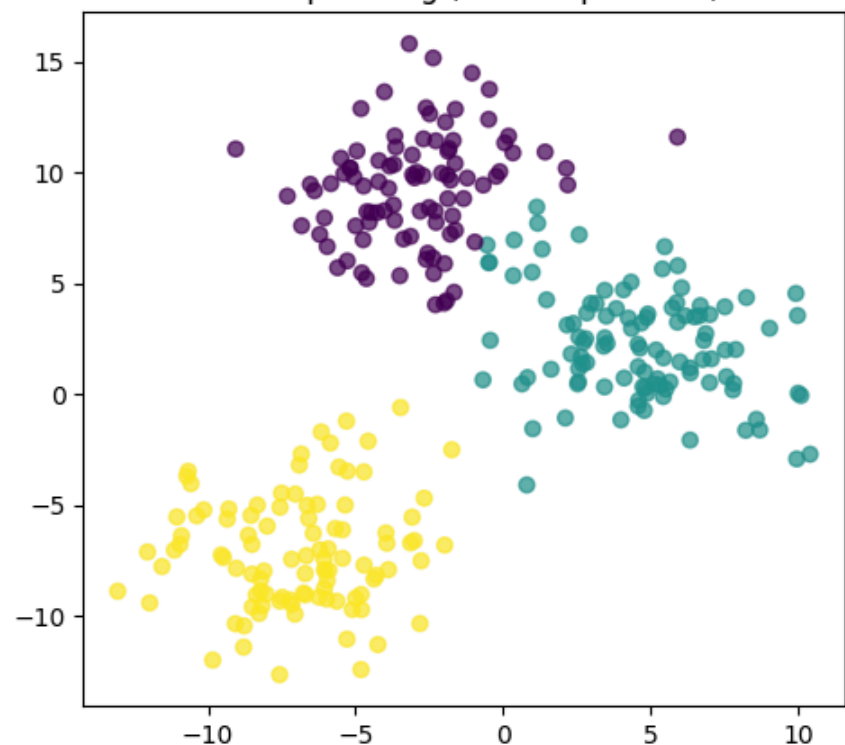
# نمودار دوم: LabelSpreading خوشه‌بندی نیمه‌نظارتی با
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=lp_labels, cmap='viridis', alpha=0.7)
plt.title('LabelSpreading (Semi-supervised)')

plt.show()
```

Spectral Clustering (No Semi-supervised Knowledge)



LabelSpreading (Semi-supervised)



  تحلیل خوشه‌بندی نیمه‌نظارتی با LabelSpreading

در این مثال، هدف مقایسه دو رویکرد خوشه‌بندی روی داده‌های مصنوعی چالش‌برانگیز با همپوشانی بالا است:

- داده‌ها با **انحراف معیار بالا** تولید شده‌اند تا خوشه‌ها به هم نزدیک و تفکیک‌ناپذیر باشند.
- ۹۰٪ از برچسب‌ها حذف شده‌اند تا شرایط **یادگیری نیمه‌نظارتی** شبیه‌سازی شود.
- در حالت اول، الگوریتم **Spectral Clustering** بدون هیچ دانش برچسبی اجرا شده و صرفاً بر اساس ساختار داده خوشه‌بندی انجام داده است.
- در حالت دوم، الگوریتم **LabelSpreading** با استفاده از تعداد کمی برچسب، اطلاعات را در میان داده‌ها منتشر کرده و خوشه‌بندی بهتری ارائه داده است.

مقایسه دو نمودار نشان می‌دهد که حتی با مقدار کمی داده برچسب‌دار، می‌توان با استفاده از روش‌های نیمه‌نظارتی مانند **LabelSpreading**، خوشه‌بندی دقیق‌تری نسبت به روش‌های کاملاً بدون نظارت به دست آورد. این رویکرد در کاربردهایی مانند تحلیل داده‌های پزشکی، تشخیص رفتار، و دسته‌بندی متون بسیار مؤثر است.

The End of Chapter 9